```
// ============ FILE: main.tsx ============
// renderer/src/main.tsx
// 渲染进程入口：创建 React 根节点并挂载应用。
// 演讲者窗口使用同一入口与预加载桥接，但只加载只读演讲者视图，不加载编辑器。
import React from 'react'
import { createRoot } from 'react-dom/client'
import App from './App'
import 演讲者窗口根 from './ppt/playback/PresenterView'
import './styles.css'
import './styles/nativeControls.css'
import './styles/actionColors.css'
import './styles/flatDesktop.css'
import '../../main/ui/dialog.css'

const container = document.getElementById('root')

// 根节点缺失属于致命配置错误，直接抛出明确的中文异常，避免出现静默空白窗口
if (!container) {
  throw new Error('渲染入口初始化失败：未找到根节点 #root，请检查 renderer/index.html')
}

const 是演讲者窗口 = typeof location !== 'undefined' &&
  new URLSearchParams(location.search).get('seal-presenter') !== null

createRoot(container).render(
  <React.StrictMode>
    {是演讲者窗口 ? <演讲者窗口根 /> : <App />}
  </React.StrictMode>
)

// ============ FILE: App.tsx ============
// 应用外壳：按当前模块装配首页视图或编辑器视图，并统一挂载顶栏、侧栏与状态栏。
import React, { useEffect, useState } from 'react'
import CollaborationSaveBanner from './collaboration/CollaborationSaveBanner'
import CollaborationLinkEntry from './collaboration/CollaborationLinkEntry'
import { App as AntdApp, ConfigProvider, Modal, theme as antdTheme } from 'antd'
import zhCN from 'antd/locale/zh_CN'
import enUS from 'antd/locale/en_US'
import { AppProvider, useAppStore } from './store'
import { MODULES } from './routes'
import { NEW_DOC_NAMES } from './navConfig'
import TitleBar from './components/TitleBar'
import Sidebar from './components/Sidebar'
import HomeRail from './components/HomeRail'
import HomeSidebar from './components/HomeSidebar'
import RecommendPanel from './components/RecommendPanel'
import HomePage from './pages/HomePage'
import { 基准文件名, 注册最近文档错误弹窗, 通过对话框打开文件, 通过路径打开文件 } from './fileOpen'
```

```
import { 分组拖入文件, 在局部投放区, 携带文件, 读取拖入路径 } from './fileDrop'
import { 桥接 } from './ipc/bridge'
import type { 保存全部应答 } from './ipc/bridge'
import { 保存全部文档 } from './editor/saveAll'
import type { DocItem } from './mock/recentDocs'
import HelpManual from './components/HelpManual'
import AboutDialog from './components/AboutDialog'
import Footer from './components/Footer'
import GlobalTabs from './components/GlobalTabs'
import AiAssistant from './assistant/AiAssistant'
import NewFileNotifier, { 本机通知事件名, type 本机通知 } from './components/NewFileNotifier'
import AssociatedFileOpener from './components/AssociatedFileOpener'
import DefaultAppPrompt from './components/DefaultAppPrompt'
import ErrorBoundary from './components/ErrorBoundary'
import {CloudProvider,useCloud} from './cloud/CloudProvider'
import CloudAutosave from './cloud/CloudAutosave'
import {flushCloudBeforeClose} from './cloud/autosaveLifecycle'
import './cloud/cloud.css'
import { useIconPlatform } from './components/IconPlatform'
import { SettingsErrorFeedback, SettingsProvider, useSettings } from './store/settingsStore'
import { InterfaceLanguageAdapter } from './i18n/interfaceLanguage'

/** 动态 Ant Design 主题配置 */
const 主题配置 = ({ 深色, 移动 }: { 深色: boolean; 移动: boolean }) => ({
  algorithm: 深色 ? antdTheme.darkAlgorithm : antdTheme.defaultAlgorithm,
  token: {
    colorPrimary: 深色 ? '#4A90E2' : '#2B6CF6',
    colorText: 深色 ? '#E8EAED' : '#1A1D24',
    colorTextSecondary: 深色 ? '#9AA0A6' : '#5C6472',
    colorBgContainer: 深色 ? '#252A33' : '#FFFFFF',
    colorBgElevated: 深色 ? '#2D333F' : '#FFFFFF',
    borderRadius: 移动 ? 6 : 4,
    boxShadow: 移动 ? '0 6px 16px rgba(0,0,0,.08)' : '0 3px 12px rgba(30,38,50,.08)',
    boxShadowSecondary: '0 4px 16px rgba(30,38,50,.09)',
    fontFamily: '"Microsoft YaHei", "PingFang SC", Arial, sans-serif',
  },
})

const 外壳 = ({ 跨端首页 }: { 跨端首页?: React.ComponentType<{children:React.ReactNode}> }) => {
  // 使用 App 上下文中的 message，使提示能沿用 ConfigProvider 的中文语言包与主题
  const { message, modal } = AntdApp.useApp()
  useEffect(() => 注册最近文档错误弹窗((选项) => { modal.warning(选项) }), [modal])
  const { module, navKey, handleNav, docs, documents, activeDocumentId, PDF待预览, pdfDocuments, 保
  const { 主题, 切换主题, 语言, 设置语言 } = useSettings()
  const cloud = useCloud()

  const 是首页 = module === 'home'
  // 帮助手册以独立弹窗打开，不再切换整页
```

```
const [帮助弹窗打开, set帮助弹窗打开] = useState(false)
// 关于我们弹窗
const [关于弹窗打开, set关于弹窗打开] = useState(false)
const [通知弹窗打开, set通知弹窗打开] = useState(false)
const [客服弹窗打开, set客服弹窗打开] = useState(false)
const [本次通知, set本次通知] = useState<Array<本机通知 & { 标识: number; 时间: string }>>([])
const 通知序号 = React.useRef(0)
const [模板库打开, set模板库打开] = useState(false)
const [新建Access请求, set新建Access请求] = useState(0)
const 当前模块 = MODULES[module]
const 星标数 = docs.filter((文档) => 文档.starred).length

// 编辑器顶栏展示实际打开的文档名；尚未打开具体文档时展示该模块的默认文件名
const 当前文档名 = React.useMemo(() => {
  if (module === 'home') {
    return undefined
  }
  if (module === 'pdf') return PDF待预览?.名称 ?? NEW_DOC_NAMES.pdf
  const 已打开 = documents.find((文档) => 文档.id === activeDocumentId)
  return 已打开?.name ?? NEW_DOC_NAMES[module]
}, [module, documents, activeDocumentId, PDF待预览])

const 页面组件 = 当前模块.page

useEffect(() => {
  if (!备份恢复提示) return
  清除备份提示()
}, [备份恢复提示, 清除备份提示])

useEffect(() => {
  const 记录通知 = (事件: Event) => {
    const 通知 = (事件 as CustomEvent<本机通知>).detail
    if (!通知 || typeof 通知.标题 !== 'string' || typeof 通知.内容 !== 'string') return
    通知序号.current += 1
    set本次通知((当前) => [{ ...通知, 标识: 通知序号.current, 时间: new Date().toLocaleTimeString(语言, { hour:
  }
  window.addEventListener(本机通知事件名, 记录通知)
  return () => window.removeEventListener(本机通知事件名, 记录通知)
}, [语言])

const 打开本地文件 = () => {
  void 通过对话框打开文件(message, modal, (类型, 内容, 路径, 警告, 页面设置, 文件指纹) => createDoc(类型, 内容, { 路径, 警告,
    .then((成功) => { if (成功) refreshRecents() })
}

// 拖入文件打开：编辑区自身处理的拖放（插入图片、调整幻灯片顺序等）会先调用 preventDefault，这里不再接管。
const [拖入中, set拖入中] = useState(false)
const 拖入层级 = React.useRef(0)
```

```
  const 最新拖入打开 = React.useRef<(路径列表: string[]) => Promise<void>>(async () => {})
  最新拖入打开.current = async (路径列表: string[]) => {
    const { 可打开, 不支持 } = 分组拖入文件(路径列表)
    if (不支持.length > 0) {
      modal.warning({
        title: '部分文件无法打开',
        content: `暂不支持以下格式：${不支持.map(基准文件名).join('、')}。可打开 DOCX、XLSX、PPTX、PDF、CSV、TXT、MD、HTML 等
        okText: '确定',
      })
    }
    let 已打开 = 0
    for (const 路径 of 可打开) {
      if (await 通过路径打开文件(路径, message, modal, (类型, 内容, 文件路径, 警告, 页面设置, 文件指纹) =>
        createDoc(类型, 内容, { 路径: 文件路径, 警告, 页面设置, 文件指纹 }))) 已打开 += 1
    }
    if (已打开 > 0) {
      refreshRecents()
      if (已打开 > 1) message.success(`已打开 ${已打开} 个文件`)
    }
  }
  useEffect(() => {
    // 落在编辑区自己的投放区（幻灯片舞台等）里时，交给该区域处理，窗口级拖入不介入
    const 交给局部 = (事件: DragEvent) => 在局部投放区(事件.target)
    const 进入 = (事件: DragEvent) => {
      if (!携带文件(事件.dataTransfer) || 交给局部(事件)) return
      拖入层级.current += 1
      set拖入中(true)
    }
    const 离开 = (事件: DragEvent) => {
      if (!携带文件(事件.dataTransfer) || 交给局部(事件)) return
      拖入层级.current = Math.max(0, 拖入层级.current - 1)
      if (拖入层级.current === 0) set拖入中(false)
    }
    const 经过 = (事件: DragEvent) => {
      if (!携带文件(事件.dataTransfer)) return
      if (交给局部(事件)) {
        拖入层级.current = 0
        set拖入中(false)
        return
      }
      // 不阻止默认行为就收不到 drop；同时避免窗口把文件当作网页导航打开
      事件.preventDefault()
      if (事件.dataTransfer) 事件.dataTransfer.dropEffect = 'copy'
    }
    const 放下 = (事件: DragEvent) => {
      if (!携带文件(事件.dataTransfer)) return
      拖入层级.current = 0
      set拖入中(false)
```

```
    // 编辑区内的拖放（插入图片、调整幻灯片顺序）会先调用 preventDefault，这里不接管
    if (事件.defaultPrevented || 交给局部(事件)) return
    事件.preventDefault()
    const 路径列表 = 读取拖入路径(事件.dataTransfer)
    if (路径列表.length === 0) {
      modal.warning({ title: '无法打开拖入的内容', content: '只能打开本机磁盘上的文件，请从资源管理器拖入。', okText: '确定' })
      return
    }
    void 最新拖入打开.current(路径列表)
  }
  window.addEventListener('dragenter', 进入)
  window.addEventListener('dragleave', 离开)
  window.addEventListener('dragover', 经过)
  window.addEventListener('drop', 放下)
  return () => {
    window.removeEventListener('dragenter', 进入)
    window.removeEventListener('dragleave', 离开)
    window.removeEventListener('dragover', 经过)
    window.removeEventListener('drop', 放下)
  }
}, [modal])

// 关闭时先等待最新工作区备份，再答复当前标签状态；平时的异步上报仅供核验失败时提示先前风险。
const 风险文档数 = workspaceTabs.filter((标签) => 标签.dirty).length
const 最新风险文档数 = React.useRef(风险文档数)
最新风险文档数.current = 风险文档数
const 最新备份刷新 = React.useRef(刷新工作状态备份)
最新备份刷新.current = 刷新工作状态备份
useEffect(() => {
  if (!桥接.可用) return
  return 桥接.onCloseStateRequested((标识) => {
    void (async () => {
      let 备份: { 成功: boolean; 错误?: string }
      try { 备份 = await 最新备份刷新.current(); await flushCloudBeforeClose() }
      catch (错误) { 备份 = { 成功: false, 错误: 错误 instanceof Error ? 错误.message : '无法刷新工作状态备份' } }
      const 结果 = await 桥接.respondCloseState(标识, {
        未保存数量: 最新风险文档数.current,
        备份成功: 备份.成功,
        ...(备份.错误 ? { 备份错误: 备份.错误 } : {}),
      })
      if (!结果.成功) modal.error({ title: '关闭保护不可用', content: 结果.错误 || '无法确认当前文档状态' })
    })().catch((错误: unknown) => {
      modal.error({ title: '关闭保护不可用', content: 错误 instanceof Error ? 错误.message : '无法确认当前文档状态
    })
  })
}, [modal])

// “保存后退出”：把每个未保存文档写回原路径（没有路径的先询问位置），并把结果回报给主进程。
```

```
const 最新保存全部 = React.useRef<() => Promise<保存全部应答>>(async () => ({ 成功: true, 已保存: [], 失败: [],
最新保存全部.current = async () => {
  const 待保存标签 = workspaceTabs.filter((标签) => 标签.dirty)
  const 待保存 = 待保存标签.map((标签) => {
    const 文档 = documents.find((项) => 项.id === 标签.id)
    return {
      标识: 标签.id,
      名称: 标签.name,
      类型: 标签.type as 'word' | 'table' | 'ppt' | 'pdf',
      路径: 标签.path,
      ...(文档 === undefined ? {} : {
        html: 文档.html,
        页面设置: 文档.页面设置,
        文件指纹: 文档.文件指纹,
        来源路径: 文档.来源路径,
        警告: 文档.警告,
      }),
      ...(标签.type === 'pdf' ? {PDF数据:pdfDocuments.find(d=>d.id===标签.id)?.data||undefined} : {
      ...(标签.type === 'table' ? { 表格模型: 表格文档模型[标签.id] } : {}),
      ...(标签.type === 'ppt' ? { 演示模型: 演示文档模型[标签.id] } : {}),
    }
  })
  const 结果 = await 保存全部文档(待保存)
  for (const 项 of 结果.已保存) {
    const 标签 = 待保存标签.find((当前) => 当前.id === 项.标识)
    const 文档 = documents.find((当前) => 当前.id === 项.标识)
    if (标签?.type === 'pdf') { 保存PDF文档(项.标识,项.路径,pdfDocuments.find(d=>d.id===项.标识)?.data||'');
    if (标签?.type === 'table') markDocumentSaved(项.标识, '', JSON.stringify(表格文档模型[项.标识] ?? []))
    else if (标签?.type === 'ppt') markDocumentSaved(项.标识, '', JSON.stringify(演示文档模型[项.标识]))
    else markDocumentSaved(项.标识, 文档?.html ?? '', undefined, { 页面设置: 文档?.页面设置 })
    set文档路径(项.标识, 项.路径)
    if (项.文件指纹) 更新文件指纹(项.标识, 项.文件指纹)
  }
  if (结果.已保存.length > 0) message.success(`已保存 ${结果.已保存.length} 个文档`)
  return {
    成功: 结果.失败.length === 0 && !结果.已取消,
    已保存: 结果.已保存.map((项) => 项.名称),
    失败: 结果.失败.map((项) => ({ 名称: 项.名称, 原因: 项.原因 })),
    已取消: 结果.已取消,
  }
}
useEffect(() => {
  if (!桥接.可用) return
  return 桥接.onSaveAllRequested((标识) => {
    void (async () => {
      let 应答: 保存全部应答
      try { 应答 = await 最新保存全部.current() }
      catch (错误) {
```

```
        应答 = { 成功: false, 已保存: [], 失败: [{ 名称: '当前窗口', 原因: 错误 instanceof Error ? 错误.message :
      }
      const 回报 = await 桥接.respondSaveAll(标识, 应答)
      if (!回报.成功) modal.error({ title: '保存后退出不可用', content: 回报.错误 || '无法回报保存结果' })
    })()
  })
}, [modal])
useEffect(() => {
  if (!桥接.可用) return
  void 桥接.reportUnsavedCount(风险文档数).then((结果) => {
    if (!结果.成功) modal.error({ title: '关闭保护不可用', content: 结果.错误 || '无法向主进程同步文档状态' })
  }).catch((错误: unknown) => {
    modal.error({ title: '关闭保护不可用', content: 错误 instanceof Error ? 错误.message : '无法向主进程同步文档状态
  })
}, [风险文档数, modal])

// 特殊处理：设置、帮助、PDF 页面的渲染（不需要 editor 布局）
let 主内容
if (module === 'settings') {
  主内容 = React.createElement(
    'main',
    { className: 'wps-main wps-main--settings' },
    React.createElement(ErrorBoundary, null, React.createElement(页面组件, null))
  )
} else if (module === 'help') {
  主内容 = React.createElement(
    'main',
    { className: 'wps-main wps-main--help' },
    React.createElement(ErrorBoundary, null, React.createElement(页面组件, null))
  )
} else if (module === 'pdf') {
  主内容 = React.createElement(
    'main',
    { className: 'wps-main wps-main--pdf' },
    React.createElement(ErrorBoundary, null, React.createElement(页面组件, null))
  )
} else {
  主内容 = React.createElement(
    'main',
    { className: 是首页 ? 'wps-main' : 'wps-main wps-main--editor' },
    // 页面级兜底：单个页面异常时仍保留顶栏与侧栏可用
    React.createElement(ErrorBoundary, null, 是首页
      ? React.createElement(HomePage, {
          模板库打开,
          关闭模板库: () => set模板库打开(false),
          新建Access请求,
          on新建Access已处理: () => set新建Access请求(0),
        })
```

```
      : React.createElement(页面组件, null))
  )
}

return React.createElement(
  'div',
  { className: 'wps-app', 'data-module': module },
  React.createElement(TitleBar, {
    pageName: 当前模块.label,
    documentName: 当前文档名,
    onHelp: () => set帮助弹窗打开(true),
    onShowSettings: showSettings,
    onShowHelp: () => set帮助弹窗打开(true),
    onCloud: cloud.configure,
    onLogin: cloud.configure,
    homeMode: 是首页,
    搜索词,
    on搜索变化: set搜索词,
    on通知: () => set通知弹窗打开(true),
    on客服: () => set客服弹窗打开(true),
    on关于: () => set关于弹窗打开(true),
    主题深色: 主题 === '深色',
    on切换主题: 切换主题,
    语言,
    on切换语言: 设置语言,
  }),
  React.createElement(CloudAutosave),
  React.createElement(CollaborationLinkEntry),
  React.createElement(CollaborationSaveBanner),
  是首页
    ? React.createElement(
        'div',
        { className: 'wps-homebody' },
        React.createElement(HomeRail, {
          activeKey: navKey,
          onNotify: (文本: string) => message.info(文本),
          onNavigate: (键: string) => handleNav(键, (文本: string) => message.info(文本)),
        }),
        React.createElement(HomeSidebar, {
          navKey,
          onNavigate: (键: string) => handleNav(键, (文本: string) => message.info(文本)),
          onNewDoc: (类型: 'word' | 'table' | 'ppt' | 'pdf') => createDoc(类型),
          onNewAccess: () => {
            set新建Access请求((当前) => 当前 + 1)
            handleNav('database', (文本: string) => message.info(文本))
          },
          onOpenTemplate: () => set模板库打开(true),
          onOpenFile: () => {
```

```
            // 与首页空状态共用同一条打开链路
            打开本地文件()
          },
          onOpenPdf: () => {
            createDoc('pdf')
          },
          onOpenHelp: () => set帮助弹窗打开(true),
          onNotify: (文本: string) => message.info(文本),
        }),
        React.createElement('main', { className: 'wps-home-main' }, 跨端首页 ? React.createElemen
        React.createElement(RecommendPanel, {
          onOpenFile: () => {
            打开本地文件()
          },
          onOpenPdf: () => createDoc('pdf'),
          onNewDoc: (类型: 'word' | 'table' | 'ppt') => createDoc(类型),
        })
      )
    : React.createElement(
    'div',
    { className: 'wps-body' },
    (module === 'settings' || module === 'help') ? React.createElement(Sidebar, {
      mode: 是首页 ? 'home' : (module === 'settings' ? 'settings' : module === 'help' ? 'help' :
      activeKey: navKey,
      onSelect: (键: string) => {
        if (键 === 'settings') {
          showSettings()
        } else if (键 === 'help') {
          showHelp()
        } else if (键 === 'pdf') {
          createDoc('pdf')
        } else {
          handleNav(键, (文本: string) => message.info(文本))
        }
      },
      moduleLabel: 当前模块.label,
      onBack: goHome,
      onShowSettings: showSettings,
      onShowHelp: () => set帮助弹窗打开(true),
      onShowPdf: () => createDoc('pdf'),
    }) : null,
    React.createElement('div', { style: { display:'flex',flexDirection:'column',flex:1,minWid
),
React.createElement(GlobalTabs, null),
React.createElement(Footer, { total: docs.length, starred: 星标数 }),
React.createElement(Modal, {
  open: 客服弹窗打开, onCancel: () => set客服弹窗打开(false),
  onOk: () => set客服弹窗打开(false), centered: true, width: 480,
```

```
  title: '联系客服', okText: '关闭', cancelButtonProps: { style: { display: 'none' } },
}, React.createElement('div', { className: 'seal-contact' },
  React.createElement('p', null, '如需反馈问题或咨询，请联系：'),
  React.createElement('dl', null,
    React.createElement('dt', null, '联系邮箱'), React.createElement('dd', null, '24519660@qq.c
  React.createElement('p', { className: 'seal-contact__hint' }, '反馈问题时请附上软件版本、操作步骤和相关截图，方便定
React.createElement(
  Modal,
  {
    open: 帮助弹窗打开,
    onCancel: () => set帮助弹窗打开(false),
    footer: null,
    width: 920,
    centered: true,
    title: '帮助手册',
    styles: { body: { maxHeight: '68vh', overflowY: 'auto', paddingTop: 0 } },
  },
  React.createElement(HelpManual, {
    打开状态: 帮助弹窗打开,
    关闭回调: () => set帮助弹窗打开(false),
  })
),
React.createElement(AboutDialog, {
  打开状态: 关于弹窗打开,
  关闭回调: () => set关于弹窗打开(false),
}),
React.createElement(
  Modal,
  {
    open: 通知弹窗打开,
    onCancel: () => set通知弹窗打开(false),
    footer: null,
    width: 520,
    title: '通知',
    styles: { body: { maxHeight: '60vh', overflowY: 'auto' } },
  },
  本次通知.length === 0
    ? React.createElement('p', { className: 'wps-notice-empty' }, '本次运行暂无通知')
    : React.createElement('ul', { className: 'wps-notice-list' }, 本次通知.map((通知) => React.cr
        'li', { className: 'wps-notice-list__item', key: 通知.标识 },
        React.createElement('div', { className: 'wps-notice-list__head' },
          React.createElement('strong', null, 通知.标题),
          React.createElement('time', null, 通知.时间)),
        React.createElement('p', null, 通知.内容)
      )))
),
拖入中 ? React.createElement('div', { className: 'wps-drop-overlay', role: 'status', 'aria-liv
  React.createElement('div', { className: 'wps-drop-overlay__panel' },
```

```
        React.createElement('strong', null, '松开鼠标即可打开文件'),
        React.createElement('span', null, '支持 DOCX、XLSX、PPTX、PDF、CSV、TXT、MD、HTML，可一次拖入多个'))) :
    React.createElement(AiAssistant, null),
    React.createElement(NewFileNotifier, null),
    React.createElement(AssociatedFileOpener, null),
    React.createElement(DefaultAppPrompt, null)
  )
}

/** 在当前主题下创建组件库上下文，使弹窗和表单也随主题切换。 */
export const 动态主题容器 = ({ children }: { children: React.ReactNode }) => {
  const { 主题, 语言 } = useSettings()
  return React.createElement(ConfigProvider, {
    locale: 语言 === 'en-US' ? enUS : zhCN,
    theme: 主题配置({ 深色: 主题 === '深色', 移动: useIconPlatform() === 'mobile' }),
    button: { autoInsertSpace: false },
  }, children)
}

const App = ({ 初始最近文档, 跨端导航, 跨端首页 }: { 初始最近文档?: DocItem[]; 跨端导航?: React.ComponentType; 跨端首页?: R
  React.createElement(
    SettingsProvider,
    null,
    React.createElement(
      动态主题容器,
      null,
      React.createElement(
        AntdApp,
        null,
        React.createElement(InterfaceLanguageAdapter, null),
        React.createElement(SettingsErrorFeedback, null),
        React.createElement(
          ErrorBoundary,
          null,
          React.createElement(CloudProvider, { children: React.createElement(AppProvider, { 初始最
        )
      )
    )
  )

export default App

// ============ FILE: store.ts ============
// 应用状态层：集中承载当前模块、导航筛选、视图模式、排序方式、文档列表与编辑器文档。
// 采用 Context 而非全局变量，保证状态随组件树卸载而释放，便于测试隔离。
import React, { createContext, useContext, useCallback, useEffect, useMemo, useRef, useState }
import { App as AntdApp, Modal } from 'antd'
import {
```

```
  filterDocs,
  sortDocs,
  type DocItem,
  type SortKey,
} from './mock/recentDocs'
import { DOC_TYPE_TO_MODULE, NAV_GROUPS, NEW_DOC_NAMES, type ModuleKey } from './navConfig'
import { 桥接, type 演示资源条目 } from './ipc/bridge'
import { 基准文件名, 记录最近文档, 读取本地文件内容 } from './fileOpen'
import { 创建工作表, 检查工作簿更新, 重算工作簿, type Sheet } from './sheet/model'
import { 从Html表格构建工作表, type Xlsx工作表元数据 } from './sheet/sheetImport'
import { 创建演示文稿, type 演示文稿 } from './ppt/deck'
import { 迁移演示文稿, 演示内容快照, 收集演示资源标识, 校验演示备份资源 } from './ppt/model/migrations'
import { 页面设置相同, type 文字页面设置 } from './office/docModel'

export type ViewMode = 'grid' | 'list'

/** 当前浏览页码属于工作区状态，不改变 PPTX 的正文内容。损坏快照仍视为未保存。 */
function 演示内容相同(当前: 演示文稿 | undefined, 已保存?: string): boolean {
  if (!当前 || !已保存) return false
  try {
    return 演示内容快照(迁移演示文稿(当前)) === 演示内容快照(迁移演示文稿(JSON.parse(已保存)))
  } catch { return false }
}

/** 各编辑器文档的身份与文字内容；表格和演示模型按相同标识独立保存。 */
export interface EditorDocument {
  id: string
  name: string
  html: string
  /** 最近一次成功写盘的文字内容，跨页面切换及自动备份后仍可判断未保存修改。 */
  已保存Html: string
  /** 最近一次成功写盘的结构化模型，供表格和演示判断修改状态。 */
  已保存模型?: string
  页面设置?: 文字页面设置
  已保存页面设置?: 文字页面设置
  type?: 'word' | 'table' | 'ppt'
  来源路径?: string
  文件指纹?: string
  警告?: string[]
}

export interface WorkspaceTab {
  id: string
  name: string
  type: 'word' | 'table' | 'ppt' | 'pdf'
  path: string | null
  dirty: boolean
}
```

```
export interface PdfDocument {
  id: string
  name: string
  path: string | null
  data: string | null
  dirty?: boolean
}

type 备份结果 = { 成功: boolean; 错误?: string }
type 备份任务 = { 操作: '保存'; 内容: string } | { 操作: '清理' }

export interface AppState {
  /** 当前模块，home 为首页，其余为编辑器 */
  module: ModuleKey
  setModule: (模块: ModuleKey) => void
  /** 首页内的导航筛选键 */
  navKey: string
  setNavKey: (键: string) => void
  /** 顶栏搜索关键词：按文档名过滤最近列表 */
  搜索词: string
  set搜索词: (词: string) => void
  /** 重新读取持久化的最近文档 */
  refreshRecents: () => void
  最近读取错误: string | null
  清除最近读取错误: () => void
  viewMode: ViewMode
  setViewMode: (模式: ViewMode) => void
  sortKey: SortKey
  setSortKey: (键: SortKey) => void
  /** 首页展示用的文档列表 */
  docs: DocItem[]
  /** 经筛选与排序后用于渲染的文档 */
  visibleDocs: DocItem[]
  toggleStar: (标识: string) => Promise<void>
  /** 首页卡片的选中标识 */
  activeDocId: string | null
  setActiveDocId: (标识: string | null) => void
  /** 打开首页文档：进入对应模块并新建编辑器标签 */
  openDoc: (文档: DocItem) => Promise<void>
  /** 新建文档：进入对应模块并创建编辑器标签；初始内容对 word 为 HTML，对表格为 HTML 表格，对演示为演示文稿模型 */
  createDoc: (类型: DocItem['type'], 初始内容?: unknown, 来源?: { 路径?: string; 名称?: string; 警告?: string
  表格文档模型: Record<string, Sheet[]>
  更新表格文档模型: (标识: string, 更新: React.SetStateAction<Sheet[]>) => void
  演示文档模型: Record<string, 演示文稿>
  设置演示历史资源: (标识: string, 引用: string[]) => void
  更新演示文档模型: (标识: string, 更新: React.SetStateAction<演示文稿>) => void
  PDF待预览: { 路径: string; 名称: string; 数据: string } | null
```

```
  pdfDocuments: readonly PdfDocument[]
  更新PDF内容: (标识:string, 数据:string) => void
  保存PDF文档: (标识:string, 路径:string, 数据:string) => void
  /** 重命名文档；传入纯空白名称时不生效，避免写入无效文件名 */
  renameDoc: (标识: string, 名称: string) => Promise<void>
  /** 删除文档；若删除的是当前打开文档，同时清空选中标识 */
  removeDoc: (标识: string) => Promise<void>
  /** 批量从最近列表移除；逐条持久化，返回实际移除与失败的条目，不删除磁盘文件 */
  removeDocs: (标识列表: string[]) => Promise<{ 已移除: string[]; 失败: Array<{ 标识: string; 名称: string;
  /** 处理首页导航点击：未实现项给出中文提示，不切换内容 */
  handleNav: (键: string, 提示: (文本: string) => void) => void
  /** 编辑器已打开的文档 */
  documents: EditorDocument[]
  /** 编辑器当前文档标识 */
  activeDocumentId: string | null
  setActiveDocumentId: (标识: string | null) => void
  updateEditorHtml: (标识: string, html: string) => void
  更新文字页面设置: (标识: string, 设置: 文字页面设置 | undefined) => void
  markDocumentSaved: (标识: string, html: string, 模型快照?: string, 文字保存快照?: { 页面设置?: 文字页面设置 }) => v
  文档路径: Record<string, string | null>
  set文档路径: (标识: string, 路径: string | null) => void
  更新文件指纹: (标识: string, 指纹: string) => void
  /** 写盘前查询目标路径是否被另一个工作区标签占用；调用方负责显示友好弹窗。 */
  查找保存路径占用: (当前标识: string | null, 目标路径: string) => WorkspaceTab | null
  closeEditorDoc: (标识: string) => void
  /** 全局底部工作区标签，顺序与文件打开顺序一致；首页单独固定在首位。 */
  workspaceTabs: WorkspaceTab[]
  activeWorkspaceTabId: string | null
  selectWorkspaceTab: (标识: string) => void
  closeWorkspaceTab: (标识: string) => void
  /** 在当前模块下新建一个编辑器标签 */
  createEditorDoc: () => void
  /** 自动备份恢复的提示文案；为 null 表示无提示 */
  备份恢复提示: string | null
  /** 启动时工作区恢复已结束；关联文件应在此后进入标签，避免被恢复状态覆盖。 */
  启动恢复结束: boolean
  /** 关闭前取消防抖，等待已发出的写入，再将最新工作区写盘。 */
  刷新工作状态备份: () => Promise<备份结果>
  清除备份提示: () => void
  /** 切换到设置页面 */
  showSettings: () => void
  /** 切换到帮助页面 */
  showHelp: () => void
  /** 返回首页 */
  goHome: () => void
}

const AppContext = createContext<AppState | null>(null)
```

```
/** 把时间戳格式化为首页展示用的 YYYY-MM-DD HH:mm */
function 格式化时间(时间戳?: number): string {
  if (时间戳 === undefined || !Number.isFinite(时间戳)) {
    const 现在 = new Date()
    const 补零 = (数值: number) => String(数值).padStart(2, '0')
    return `${现在.getFullYear()}-${补零(现在.getMonth() + 1)}-${补零(现在.getDate())} ${补零(现在.getHours()
  }
  const 日期 = new Date(时间戳)
  const 补零 = (数值: number) => String(数值).padStart(2, '0')
  return `${日期.getFullYear()}-${补零(日期.getMonth() + 1)}-${补零(日期.getDate())} ${补零(日期.getHours())}
}

/** 生成文档标识；使用时间戳与随机后缀，避免同一毫秒内重复 */
function 生成文档标识(): string {
  return `doc-${Date.now()}-${Math.random().toString(36).slice(2, 8)}`
}

/** 取模块对应的默认文档名，home 不参与编辑器 */
function 模块默认文档名(模块: ModuleKey): string {
  return 模块 === 'home' ? '未命名文档.docx' : NEW_DOC_NAMES[模块]
}

function 是同一路径(左: string | null | undefined, 右: string): boolean {
  return typeof 左 === 'string' && 左.replace(/\\/g, '/').toLowerCase() === 右.replace(/\\/g, '/')
}

function 预览PDF文档(文档: PdfDocument | null | undefined): { 路径: string; 名称: string; 数据: string } |
  if (!文档?.data) return null
  return { 路径: 文档.path ?? `memory:${文档.id}`, 名称: 文档.name, 数据: 文档.data }
}

function 是工作表模型(值: unknown): 值 is Sheet {
  if (typeof 值 !== 'object' || 值 === null) return false
  const 表 = 值 as Partial<Sheet>
  return typeof 表.id === 'string' && typeof 表.name === 'string' &&
    typeof 表.单元格 === 'object' && 表.单元格 !== null &&
    Number.isInteger(表.行数) && Number.isInteger(表.列数) &&
    Array.isArray(表.列宽) && Array.isArray(表.行高) && Array.isArray(表.合并区域)
}

export function AppProvider({ children, 初始最近文档 }: { children: React.ReactNode; 初始最近文档?: DocItem
  const { modal: 上下文弹窗 } = AntdApp.useApp()
  const 弹窗 = typeof 上下文弹窗.error === 'function' ? 上下文弹窗 : Modal
  const [module, setModule] = useState<ModuleKey>('home')
  const [navKey, setNavKey] = useState<string>('home')
  const [搜索词, set搜索词状态] = useState<string>('')
  const [viewMode, setViewMode] = useState<ViewMode>('grid')
```

```
  const [sortKey, setSortKey] = useState<SortKey>('time')
  const [docs, setDocs] = useState<DocItem[]>(初始最近文档 ?? [])
  const [最近读取错误, set最近读取错误] = useState<string | null>(null)
  const [activeDocId, setActiveDocId] = useState<string | null>(null)
  const [documents, setDocuments] = useState<EditorDocument[]>([])
  const [activeDocumentId, setActiveDocumentId] = useState<string | null>(null)
  const [文档路径, set文档路径状态] = useState<Record<string, string | null>>({})
  const [表格文档模型, set表格文档模型] = useState<Record<string, Sheet[]>>({})
  const [演示文档模型, set演示文档模型] = useState<Record<string, 演示文稿>>({})
  const [演示历史资源, set演示历史资源] = useState<Record<string, string[]>>({})
  const 设置演示历史资源 = useCallback((标识: string, 引用: string[]) => {
    set演示历史资源(当前 => {
      if (JSON.stringify(当前[标识] ?? []) === JSON.stringify(引用)) return 当前
      const 更新 = { ...当前 }
      if (引用.length) 更新[标识] = 引用
      else delete 更新[标识]
      return 更新
    })
  }, [])
  const 已同步资源快照 = useRef<Set<string>>(new Set())
  const 资源同步链 = useRef<Promise<void>>(Promise.resolve())
  const 资源同步错误 = useRef<string | null>(null)
  useEffect(() => {
    // 先为真实历史建立所有权，再替换当前文稿引用，避免删除后撤销资源被回收。
    const 目标 = new Map<string, string[]>([
      ...Object.entries(演示历史资源).filter(([标识]) => 标识 === '独立文稿' || 演示文档模型[标识]).map(([标识, 引用]) =>
      ...Object.entries(演示文档模型).map(([标识, 文稿]) => [`文稿:${标识}`, 收集演示资源标识(文稿)] as const),
    ].filter(([, 引用]) => 引用.length > 0))
    const 待释放 = [...已同步资源快照.current].filter((标识) => !目标.has(标识))
    if (目标.size === 0 && 待释放.length === 0) return
    已同步资源快照.current = new Set(目标.keys())
    const 执行 = async () => {
      for (const [标识, 引用] of 目标) {
        const 结果 = await 桥接.presentationResources.sync(标识, 引用)
        if (!结果.成功) throw new Error(结果.错误 || '无法更新演示资源引用')
      }
      for (const 标识 of 待释放) {
        const 结果 = await 桥接.presentationResources.release(标识)
        if (!结果.成功) throw new Error(结果.错误 || '无法释放演示资源引用')
      }
      资源同步错误.current = null
    }
    const 本轮 = 资源同步链.current.then(执行, 执行)
    资源同步链.current = 本轮.catch((错误: unknown) => {
      资源同步错误.current = 错误 instanceof Error ? 错误.message : '演示资源无法同步'
      弹窗.error({ title: '演示资源同步失败', content: 资源同步错误.current })
    })
  }, [演示文档模型, 演示历史资源])
```

```
useEffect(() => () => {
  const 待释放 = [...已同步资源快照.current]
  资源同步链.current = 资源同步链.current.then(async () => {
    for (const 标识 of 待释放) {
      const 结果 = await 桥接.presentationResources.release(标识)
      if (!结果.成功) throw new Error(结果.错误 || '无法释放演示资源引用')
    }
  }).catch((错误: unknown) => {
    弹窗.error({ title: '演示资源释放失败', content: 错误 instanceof Error ? 错误.message : '演示资源无法释放' })
  })
}, [])
const [PDF待预览, setPDF待预览] = useState<{ 路径: string; 名称: string; 数据: string } | null>(null)
const [pdfDocuments, setPdfDocuments] = useState<PdfDocument[]>([])
const [activePdfId, setActivePdfId] = useState<string | null>(null)
const [workspaceOrder, setWorkspaceOrder] = useState<string[]>([])
const [工作状态设置, set工作状态设置] = useState<{ 值: boolean | null; 错误: string | null }>(() => {
  try { return { 值: localStorage.getItem('seal-session-restore') !== 'false', 错误: null } }
  catch (错误) { return { 值: null, 错误: 错误 instanceof Error ? 错误.message : '本机设置无法读取' } }
})
const 保存工作状态 = 工作状态设置.值

useEffect(() => {
  if (工作状态设置.错误) 弹窗.error({ title: '读取工作状态设置失败', content: `自动备份已暂停，原有备份文件保持不变。${工作状态设置.错误}` }
}, [工作状态设置.错误])

useEffect(() => {
  const 同步设置 = () => {
    try { set工作状态设置({ 值: localStorage.getItem('seal-session-restore') !== 'false', 错误: null }
    catch (错误) { set工作状态设置({ 值: null, 错误: 错误 instanceof Error ? 错误.message : '本机设置无法读取' }) }
  }
  window.addEventListener('seal-session-setting-changed', 同步设置)
  return () => window.removeEventListener('seal-session-setting-changed', 同步设置)
}, [])

const workspaceTabs = useMemo<WorkspaceTab[]>(() => {
  const 文档标签 = new Map<string, WorkspaceTab>()
  for (const 文档 of documents) {
    const 类型 = 文档.type ?? 'word'
    const 路径 = 文档路径[文档.id] ?? null
    const 当前模型 = 类型 === 'table' ? 表格文档模型[文档.id] : 类型 === 'ppt' ? 演示文档模型[文档.id] : null
    const 模型有修改 = 类型 === 'ppt'
      ? !演示内容相同(演示文档模型[文档.id], 文档.已保存模型)
      : 类型 !== 'word' && (当前模型 === undefined || JSON.stringify(当前模型) !== 文档.已保存模型)
    文档标签.set(文档.id, {
      id: 文档.id,
      name: 文档.name,
      type: 类型,
      path: 路径,
```

```
      dirty: 路径 === null || (类型 === 'word'
        ? 文档.html !== 文档.已保存Html || !页面设置相同(文档.页面设置, 文档.已保存页面设置)
        : 模型有修改),
    })
  }
  for (const 文档 of pdfDocuments) {
    文档标签.set(文档.id, { id: 文档.id, name: 文档.name, type: 'pdf', path: 文档.path, dirty: Boolean(文档
  }
  return workspaceOrder.flatMap((标识) => {
    const 标签 = 文档标签.get(标识)
    return 标签 ? [标签] : []
  })
}, [documents, 文档路径, 表格文档模型, 演示文档模型, pdfDocuments, workspaceOrder])
const workspaceTabsRef = useRef(workspaceTabs)
workspaceTabsRef.current = workspaceTabs
const activeWorkspaceTabId = module === 'home' ? 'home'
  : module === 'pdf' ? activePdfId
    : module === 'word' || module === 'table' || module === 'ppt' ? activeDocumentId : null

const visibleDocs = useMemo(() => {
  const 关键词 = 搜索词.trim().toLowerCase()
  const 按词过滤 = 关键词 === '' ? docs : docs.filter((文档) => 文档.name.toLowerCase().includes(关键词))
  return sortDocs(filterDocs(按词过滤, navKey), sortKey)
}, [docs, navKey, sortKey, 搜索词])

/** 重新读取持久化的最近文档（首页刷新按钮） */
const refreshRecents = () => {
  void 桥接.recentList().then((结果) => {
    if (!结果.成功 || !Array.isArray(结果.数据)) {
      set最近读取错误(结果.错误 || '最近文档读取失败')
      return
    }
    setDocs(转文档条目(结果.数据))
    set最近读取错误(null)
  }).catch((错误: unknown) => {
    set最近读取错误(错误 instanceof Error ? 错误.message : '最近文档读取失败')
  })
}

/** 把主进程的持久化记录转换为首页可渲染的文档条目 */
const 转文档条目 = (记录: Array<{ 路径: string; 名称: string; 类型: string; 置顶?: boolean; 时间?: number }>)
  记录
    .filter((记录) => 记录.类型 === 'word' || 记录.类型 === 'table' || 记录.类型 === 'ppt' || 记录.类型 === 'pd
    .map((记录) => ({
      id: `recent-${记录.路径}`,
      name: 记录.名称,
      type: 记录.类型 as DocItem['type'],
      size: 0,
```

```
      updatedAt: 格式化时间(记录.时间),
      starred: 记录.置顶 === true,
      shared: false,
      路径: 记录.路径,
    }))

// 挂载时读取持久化的最近文档；空记录展示空状态，不展示虚构文件
useEffect(() => {
  if (初始最近文档 === undefined && 桥接.可用) refreshRecents()
  // 仅在挂载时加载一次
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [])

// 自动备份须在旧备份读取完成后才可写入，避免启动时的空状态清除恢复文件。
const 备份计时 = useRef<number | null>(null)
const [备份就绪, set备份就绪] = useState(false)
const [启动恢复结束, set启动恢复结束] = useState(false)
const 已提示备份错误 = useRef(false)
const 备份操作链 = useRef<Promise<备份结果>>(Promise.resolve({ 成功: true }))
const 最新备份状态 = useRef({
  保存工作状态, 备份就绪, documents, activeDocumentId, 文档路径,
  表格文档模型, 演示文档模型, pdfDocuments, activePdfId, module, workspaceOrder,
})
最新备份状态.current = {
  保存工作状态, 备份就绪, documents, activeDocumentId, 文档路径,
  表格文档模型, 演示文档模型, pdfDocuments, activePdfId, module, workspaceOrder,
}

const 创建当前备份任务 = (): 备份任务 => {
  const 状态 = 最新备份状态.current
  if (!桥接.可用) throw new Error('当前环境无法保存工作状态')
  if (!状态.备份就绪) throw new Error('工作状态恢复仍在进行中，请稍后重试')
  if (状态.保存工作状态 === null) throw new Error('无法读取工作状态设置，请检查本机设置后重试')
  if (!状态.保存工作状态 || (状态.documents.length === 0 && 状态.pdfDocuments.length === 0)) return { 操作:
  return { 操作: '保存', 内容: JSON.stringify({
    documents: 状态.documents, activeDocumentId: 状态.activeDocumentId,
    文档路径: 状态.文档路径, 表格文档模型: 状态.表格文档模型,
    演示文档模型: 状态.演示文档模型,
    pdfDocuments: 状态.pdfDocuments.map(({ id, name, path, data, dirty }) => ({ id, name, path,
    activePdfId: 状态.activePdfId, activeModule: 状态.module, workspaceOrder: 状态.workspaceOrder,
  }) }
}

const 排队执行备份 = (任务: 备份任务): Promise<备份结果> => {
  const 执行 = async (): Promise<备份结果> => {
    try {
      let 内容 = 任务.操作 === '保存' ? 任务.内容 : ''
      if (任务.操作 === '保存') {
```

```
        const 数据 = JSON.parse(内容) as { 演示文档模型?: Record<string, 演示文稿>; 演示资源字节?: 演示资源条目[] }
        const 资源标识 = [...new Set(Object.values(数据.演示文档模型 ?? {}).flatMap(收集演示资源标识))]
        if (资源标识.length > 0) {
          await 资源同步链.current
          if (资源同步错误.current) throw new Error(资源同步错误.current)
          const 资源 = await 桥接.presentationResources.export(资源标识)
          if (!资源.成功 || !Array.isArray(资源.条目) || 资源.条目.length !== 资源标识.length) {
            throw new Error(资源.错误 || '演示资源字节不完整，已阻止写入不完整备份')
          }
          数据.演示资源字节 = 资源.条目
          内容 = JSON.stringify(数据)
        }
      }
      const 结果 = 任务.操作 === '清理' ? await 桥接.backupClear() : await 桥接.backupSave(内容)
      return 结果.成功 ? { 成功: true } : { 成功: false, 错误: 结果.错误 || '无法保存工作状态' }
    } catch (错误) {
      return { 成功: false, 错误: 错误 instanceof Error ? 错误.message : '无法保存工作状态' }
    }
  }
  const 本次 = 备份操作链.current.then(执行, 执行)
  备份操作链.current = 本次
  return 本次
}

const 刷新工作状态备份 = async (): Promise<备份结果> => {
  if (备份计时.current !== null) {
    window.clearTimeout(备份计时.current)
    备份计时.current = null
  }
  try {
    for (let 次数 = 0; 次数 < 3; 次数 += 1) {
      const 任务 = 创建当前备份任务()
      const 结果 = await 排队执行备份(任务)
      if (!结果.成功) return 结果
      const 最新任务 = 创建当前备份任务()
      const 内容相同 = 任务.操作 === '清理'
        ? 最新任务.操作 === '清理'
        : 最新任务.操作 === '保存' && 任务.内容 === 最新任务.内容
      if (内容相同) {
        return { 成功: true }
      }
    }
    return { 成功: false, 错误: '工作状态在关闭检查期间持续变化，请停止编辑后重试' }
  } catch (错误) {
    return { 成功: false, 错误: 错误 instanceof Error ? 错误.message : '无法刷新工作状态备份' }
  }
}
```

```
useEffect(() => {
  if (!备份就绪 || 保存工作状态 !== true || !桥接.可用) return
  if (备份计时.current !== null) {
    window.clearTimeout(备份计时.current)
  }
  备份计时.current = window.setTimeout(() => {
    备份计时.current = null
    let 任务: 备份任务
    try { 任务 = 创建当前备份任务() }
    catch (错误) {
      弹窗.error({ title: '自动备份失败', content: 错误 instanceof Error ? 错误.message : '无法创建工作状态备份' })
      return
    }
    void 排队执行备份(任务).then((结果) => {
      if (结果.成功) {
        已提示备份错误.current = false
      } else if (!已提示备份错误.current) {
        已提示备份错误.current = true
        弹窗.error({ title: '自动备份失败', content: 结果.错误 || '无法保存编辑备份，请及时手动保存文件' })
      }
    })
  }, 1500)
  return () => {
    if (备份计时.current !== null) {
      window.clearTimeout(备份计时.current)
    }
  }
}, [备份就绪, 保存工作状态, documents, activeDocumentId, 文档路径, 表格文档模型, 演示文档模型, pdfDocuments, activePdfI

useEffect(() => {
  if (!备份就绪 || 保存工作状态 !== false || !桥接.可用) return
  void 排队执行备份({ 操作: '清理' }).then((结果) => {
    if (!结果.成功) throw new Error(结果.错误 || '无法清除已保存的工作状态')
  }).catch((错误: unknown) => {
    弹窗.error({ title: '工作状态清理失败', content: 错误 instanceof Error ? 错误.message : '无法清除已保存的工作状态'
  })
}, [备份就绪, 保存工作状态])

// 挂载时检测上次异常退出留下的备份并恢复，避免内容丢失
const [备份恢复提示, set备份恢复提示] = useState<string | null>(null)
useEffect(() => {
  let 有效 = true
  const 完成恢复 = () => {
    if (!有效) return
    set备份就绪(true)
    set启动恢复结束(true)
  }
  const 保留原始备份 = async (内容?: string): Promise<string> => {
```

```
  const 结果 = await 桥接.backupPreserve(内容)
  if (!结果.成功 || typeof 结果.路径 !== 'string' || 结果.路径.length === 0) {
    throw new Error(结果.错误 || '无法确认原始备份的保留位置')
  }
  return 结果.路径
}
if (!桥接.可用 || 保存工作状态 !== true) {
  完成恢复()
  return () => { 有效 = false }
}
const 恢复工作状态 = async () => {
  let 原始内容: string | undefined
  try {
    const 结果 = await 桥接.backupLoad()
    if (!有效) return
    if (!结果.成功) throw new Error(结果.错误 || '备份读取失败')
    if (结果.内容 === null || 结果.内容 === undefined) {
      完成恢复()
      return
    }
    原始内容 = 结果.内容
    const 数据 = JSON.parse(原始内容) as {
      documents?: EditorDocument[]
      activeDocumentId?: string | null
      文档路径?: Record<string, string | null>
      表格文档模型?: Record<string, Sheet[]>
      演示文档模型?: Record<string, 演示文稿>
      演示资源字节?: 演示资源条目[]
      pdfDocuments?: Array<{ id: string; name: string; path: string | null; data?: string |
      activePdfId?: string | null
      activeModule?: ModuleKey
      workspaceOrder?: string[]
    }
    if (typeof 数据 !== 'object' || 数据 === null || Array.isArray(数据) ||
      (数据.documents === undefined && 数据.pdfDocuments === undefined) ||
      (数据.documents !== undefined && !Array.isArray(数据.documents)) ||
      (数据.pdfDocuments !== undefined && !Array.isArray(数据.pdfDocuments))) {
      throw new Error('备份结构无效，已阻止加载不完整的编辑状态')
    }
    const 文字表格演示 = 数据.documents ?? []
    const PDF元数据 = 数据.pdfDocuments ?? []
    if (文字表格演示.length === 0 && PDF元数据.length === 0) {
      完成恢复()
      return
    }
    const 有效文档 = 文字表格演示.every((文档) =>
      typeof 文档?.id === 'string' && typeof 文档.name === 'string' && typeof 文档.html === 'stri
      (文档.type === 'word' || 文档.type === undefined ||
```

```
    (文档.type === 'table' && Array.isArray(数据.表格文档模型?.[文档.id]) && 数据.表格文档模型![文档.id].leng
    (文档.type === 'ppt' && Array.isArray(数据.演示文档模型?.[文档.id]?.幻灯片列表))))
if (!有效文档) throw new Error('备份内容缺少表格或演示模型，已阻止加载不完整的编辑状态')
const 已迁移演示: Record<string, 演示文稿> = {}
for (const 文档 of 文字表格演示) {
  if (文档.type !== 'ppt') continue
  try {
    已迁移演示[文档.id] = 迁移演示文稿(数据.演示文档模型?.[文档.id])
    if (文档.已保存模型) 迁移演示文稿(JSON.parse(文档.已保存模型))
  } catch (错误) {
    throw new Error(`无法恢复「${文档.name}」：${错误 instanceof Error ? 错误.message : '演示模型无效'}`)
  }
}
const 演示引用 = [...new Set(Object.values(已迁移演示).flatMap(收集演示资源标识))]
if (演示引用.length > 0) {
  校验演示备份资源(Object.values(已迁移演示), 数据.演示资源字节)
  const 恢复结果 = await 桥接.presentationResources.restore(数据.演示资源字节!)
  if (!恢复结果.成功) throw new Error(恢复结果.错误 || '演示资源恢复失败')
}
const PDF读取结果 = await Promise.allSettled(PDF元数据.map(async (记录): Promise<PdfDocument> =>
  if (typeof 记录?.id !== 'string' || typeof 记录.name !== 'string' ||
      (记录.path !== null && typeof 记录.path !== 'string')) {
    throw new Error('PDF 标签记录格式无效')
  }
  if (记录.path === null || 记录.dirty) {
    if (typeof 记录.data === 'string') return { ...记录, data: 记录.data }
    if (记录.data === null && 记录.name === 'PDF 工具') return { ...记录, data: null }
    throw new Error(`无法恢复「${记录.name}」：备份缺少 PDF 文件内容`)
  }
  const 内容 = await 读取本地文件内容(记录.path)
  if (内容.类型 !== 'pdf' || typeof 内容.内容 !== 'string') throw new Error(`无法恢复「${记录.name}」`)
  return { ...记录, data: 内容.内容 }
}))
if (!有效) return
const 有效PDF = PDF读取结果.flatMap((项) => 项.status === 'fulfilled' ? [项.value] : [])
const 失败PDF = PDF读取结果.filter((项): 项 is PromiseRejectedResult => 项.status === 'rejected'
if (失败PDF.length > 0) {
  const 保留路径 = await 保留原始备份(原始内容)
  if (!有效) return
  弹窗.warning({
    title: '部分 PDF 标签未恢复',
    content: `${失败PDF.map((项) => 项.reason instanceof Error ? 项.reason.message : '文件读取失败
  })
}
const 当前标识 = 文字表格演示.some((文档) => 文档.id === 数据.activeDocumentId)
  ? 数据.activeDocumentId!
  : 文字表格演示[0]?.id ?? null
const 当前文档 = 文字表格演示.find((文档) => 文档.id === 当前标识)
```

```
  // 旧版备份没有保存基线时，使用不可能成为正文的哨兵值，保守提示用户检查修改。
  setDocuments(文字表格演示.map((文档) => ({
    ...文档,
    已保存Html: typeof 文档.已保存Html === 'string' ? 文档.已保存Html : '\u0000',
  })))
  setActiveDocumentId(当前标识)
  setPdfDocuments(有效PDF)
  const 当前PDF = 有效PDF.find((文档) => 文档.id === 数据.activePdfId) ?? 有效PDF[0]
  setActivePdfId(当前PDF?.id ?? null)
  const 有效标识 = new Set([...文字表格演示.map((文档) => 文档.id), ...有效PDF.map((文档) => 文档.id)])
  const 恢复顺序 = Array.isArray(数据.workspaceOrder)
    ? 数据.workspaceOrder.filter((标识) => 有效标识.has(标识))
    : []
  setWorkspaceOrder([...恢复顺序, ...[...文字表格演示.map((文档) => 文档.id), ...有效PDF.map((文档) => 文档.i
  set文档路径状态(数据.文档路径 ?? {})
  set表格文档模型(数据.表格文档模型 ?? {})
  set演示文档模型(已迁移演示)
  const 恢复模块 = 数据.activeModule === 'home' ? 'home'
    : 数据.activeModule === 'pdf' && 当前PDF ? 'pdf'
      : 当前文档?.type ?? (当前PDF ? 'pdf' : 'home')
  setModule(恢复模块)
  setNavKey(恢复模块)
  setPDF待预览(恢复模块 === 'pdf' ? 预览PDF文档(当前PDF) : null)
  set备份恢复提示(`已恢复上次工作区的 ${文字表格演示.length + 有效PDF.length} 个标签`)
  完成恢复()
} catch (错误) {
  if (!有效) return
  const 原因 = 错误 instanceof SyntaxError ? '备份文件格式无效，无法解析'
    : 错误 instanceof Error ? 错误.message : '无法读取编辑备份'
  弹窗.confirm({
    title: '工作状态恢复失败',
    content: `读取或校验上次工作区失败：${原因}。原备份尚未修改。可以重试恢复；若继续，原备份会移到独立文件，当前会话将新建备份。`,
    okText: '保留原备份并继续',
    cancelText: '重试恢复',
    onOk: async () => {
      try {
        const 保留路径 = await 保留原始备份(原始内容)
        if (!有效) return
        完成恢复()
        弹窗.info({ title: '原备份已保留', content: `原始备份已保留在：${保留路径}。当前会话可以继续编辑和自动备份。` })
      } catch (保留错误) {
        弹窗.error({ title: '保留原备份失败', content: `原备份未被覆盖，当前会话的自动备份仍已暂停。${保留错误 instanceof Er
        throw 保留错误
      }
    },
    onCancel: () => { void 恢复工作状态() },
  })
}
```

```
  }
  void 恢复工作状态()
  // 仅在挂载时恢复一次
  // eslint-disable-next-line react-hooks/exhaustive-deps
  return () => { 有效 = false }
}, [])

const toggleStar = async (标识: string): Promise<void> => {
  const 文档 = docs.find((项) => 项.id === 标识)
  if (!文档) return
  const 新状态 = !文档.starred
  if (文档.路径 !== undefined) {
    try {
      const 结果 = await 桥接.recentAdd({ 路径: 文档.路径, 名称: 文档.name, 类型: 文档.type, 置顶: 新状态, 时间: Date.
      if (!结果.成功) throw new Error(结果.错误 || '星标状态保存失败')
    } catch (错误) {
      弹窗.error({ title: '星标设置失败', content: 错误 instanceof Error ? 错误.message : '无法保存星标状态' })
      return
    }
  }
  setDocs((当前) => 当前.map((项) => 项.id === 标识 ? { ...项, starred: 新状态 } : 项))
}

/** 追加一个编辑器标签并设为当前文档。 */
const 新建标签 = (名称: string, 类型: 'word' | 'table' | 'ppt', 初始Html = '<p><br></p>', 来源?: { 路径: st
  const 新文档: EditorDocument = {
    id: 生成文档标识(),
    name: 名称,
    html: 初始Html,
    已保存Html: 初始Html,
    已保存模型,
    页面设置,
    已保存页面设置: 页面设置,
    type: 类型,
    来源路径: 来源?.路径,
    文件指纹: 来源?.文件指纹,
    警告: 来源?.警告,
  }
  setDocuments((当前) => [...当前, 新文档])
  setWorkspaceOrder((当前) => [...当前, 新文档.id])
  setActiveDocumentId(新文档.id)
  if (来源 !== undefined) set文档路径状态((当前) => ({ ...当前, [新文档.id]: 来源.路径 }))
  return 新文档
}

const createDoc = (类型: DocItem['type'], 初始内容?: unknown, 来源?: { 路径?: string; 名称?: string; 警告?:
  const 来源路径 = typeof 来源?.路径 === 'string' && 来源.路径.trim() !== '' ? 来源.路径 : null
  if (来源路径 !== null && 类型 !== 'pdf') {
```

```
  const 已开文档 = documents.find((项) => 是同一路径(文档路径[项.id], 来源路径))
  if (已开文档) {
    setActiveDocumentId(已开文档.id)
    setModule(已开文档.type ?? 'word')
    setNavKey(已开文档.type ?? 'word')
    return
  }
}
if (类型 === 'pdf') {
  if (来源 !== undefined && typeof 初始内容 !== 'string') throw new Error('PDF 文件内容无效')
  const PDF名称 = 来源路径 !== null ? 基准文件名(来源路径) : 来源?.名称?.trim()
  if (来源 !== undefined && !PDF名称) throw new Error('PDF 文件名称无效')
  const 已开PDF = 来源路径 !== null
    ? pdfDocuments.find((项) => 是同一路径(项.path, 来源路径))
    : 来源 === undefined ? pdfDocuments.find((项) => 项.path === null && 项.data === null) : und
  if (已开PDF) {
    if (来源 !== undefined) {
      setPdfDocuments((当前) => 当前.map((项) => 项.id === 已开PDF.id
        ? { ...项, name: PDF名称!, data: 初始内容 as string } : 项))
    }
    setActivePdfId(已开PDF.id)
    setPDF待预览(来源 !== undefined ? 预览PDF文档({ ...已开PDF, name: PDF名称!, data: 初始内容 as string })
    setModule('pdf')
    setNavKey('pdf')
    return
  }
  const PDF文档: PdfDocument = {
    id: 生成文档标识(),
    name: PDF名称 ?? 'PDF 工具',
    path: 来源路径,
    data: 来源 ? 初始内容 as string : null,
  }
  setPdfDocuments((当前) => [...当前, PDF文档])
  setWorkspaceOrder((当前) => [...当前, PDF文档.id])
  setActivePdfId(PDF文档.id)
  setActiveDocId(null)
  setPDF待预览(预览PDF文档(PDF文档))
  setModule('pdf')
  setNavKey('pdf')
  return
}
const 目标模块 = DOC_TYPE_TO_MODULE[类型]
const 指定名称 = 来源?.名称?.trim()
if (来源 !== undefined && 来源路径 === null && !指定名称) throw new Error('文件路径无效，无法打开文档')
if (目标模块 === 'word' && 来源 !== undefined && typeof 初始内容 !== 'string') {
  throw new Error('文字文档内容无效，文件未打开')
}
let 新表格: Sheet[] | null = null
```

```
  let 新演示: 演示文稿 | null = null
  if (目标模块 === 'table') {
    if (初始内容 === undefined || 初始内容 === null) {
      新表格 = [创建工作表('Sheet1')]
    } else if (Array.isArray(初始内容) && 初始内容.length > 0 && 初始内容.every(是工作表模型)) {
      新表格 = 初始内容 as Sheet[]
    } else {
      const 来源列表 = typeof 初始内容 === 'string'
        ? [{ 名称: 'Sheet1', html: 初始内容 }]
        : Array.isArray(初始内容) && 初始内容.length > 0 && 初始内容.every((项) => typeof 项?.名称 === 'strin
          ? 初始内容 as Array<{ 名称: string; html: string; 页面设置?: Sheet['页面设置']; 元数据?: Xlsx工作表元数据
          : null
      if (来源列表 === null) throw new Error('表格内容格式无效，文件未打开')
      const 解析列表 = 来源列表.map((项) => 从Html表格构建工作表(项.html, 项.名称, 项.页面设置, 项.元数据))
      if (解析列表.some((项) => 项 === null)) throw new Error('工作表内容解析失败，文件未打开')
      新表格 = 重算工作簿(解析列表 as Sheet[])
    }
  } else if (目标模块 === 'ppt') {
    if (初始内容 === undefined || 初始内容 === null) {
      新演示 = 创建演示文稿()
    } else if (typeof 初始内容 === 'object' && Array.isArray((初始内容 as 演示文稿).幻灯片列表)) {
      新演示 = 迁移演示文稿(初始内容)
    } else {
      throw new Error('演示文稿内容格式无效，文件未打开')
    }
  }
  setActiveDocId(null)
  setModule(目标模块)
  const 初始Html = 目标模块 === 'word' && typeof 初始内容 === 'string' ? 初始内容 : '<p><br></p>'
  const 已保存模型 = 来源 === undefined ? undefined
    : 新表格 !== null ? JSON.stringify(新表格)
      : 新演示 !== null ? JSON.stringify(新演示) : undefined
  const 有效来源 = 来源路径 !== null ? { 路径: 来源路径, 警告: 来源?.警告, 文件指纹: 来源?.文件指纹 } : undefined
  const 文档 = 新建标签(有效来源 ? 基准文件名(有效来源.路径) : 指定名称 || 模块默认文档名(目标模块), 目标模块 as 'word' | 'table' | '
  if (新表格 !== null) set表格文档模型((当前) => ({ ...当前, [文档.id]: 新表格 }))
  if (新演示 !== null) set演示文档模型((当前) => ({ ...当前, [文档.id]: 新演示 }))
}

const openDoc = async (文档: DocItem): Promise<void> => {
  if (!文档.路径) throw new Error('最近记录缺少文件路径，无法打开')
  const 路径 = 文档.路径
  const 已存在 = documents.find((项) => 是同一路径(文档路径[项.id], 路径))
  if (已存在 !== undefined) {
    if (已存在.type === 'table' && !表格文档模型[已存在.id]) throw new Error('表格编辑状态缺失，无法继续保存；请重新打开原文件')
    if (已存在.type === 'ppt' && !演示文档模型[已存在.id]) throw new Error('演示编辑状态缺失，无法继续保存；请重新打开原文件')
    setActiveDocumentId(已存在.id)
    setModule(已存在.type ?? DOC_TYPE_TO_MODULE[文档.type])
    await 记录最近文档(路径, 文档.name, 文档.type, 文档.starred)
```

```
    refreshRecents()
    return
  }
  const 内容 = await 读取本地文件内容(路径)
  createDoc(内容.类型, 内容.类型 === 'ppt' ? 内容.演示文稿 : 内容.类型 === 'table' ? (内容.工作表列表 ?? 内容.内容) : 内容.内
  setActiveDocId(文档.id)
  await 记录最近文档(路径, 文档.name, 内容.类型, 文档.starred)
  refreshRecents()
}

const createEditorDoc = () => {
  const 目标模块: ModuleKey = module === 'home' ? 'word' : module
  if (目标模块 === 'word') 新建标签(模块默认文档名(目标模块), 'word')
  else if (目标模块 === 'table' || 目标模块 === 'ppt') createDoc(目标模块)
}

const updateEditorHtml = (标识: string, html: string) => {
  setDocuments((当前) => 当前.map((项) => (项.id === 标识 ? { ...项, html } : 项)))
}

const 更新文字页面设置 = (标识: string, 设置: 文字页面设置 | undefined) => {
  setDocuments((当前) => 当前.map((项) => 项.id === 标识 && (项.type === 'word' || 项.type === undefine
    ? { ...项, 页面设置: 设置 } : 项))
}

const markDocumentSaved = (标识: string, html: string, 模型快照?: string, 文字保存快照?: { 页面设置?: 文字页面设置
  setDocuments((当前) => 当前.map((项) => {
    if (项.id !== 标识) return 项
    const 模型 = 项.type === 'table' ? 表格文档模型[标识] : 项.type === 'ppt' ? 演示文档模型[标识] : null
    return {
      ...项,
      已保存Html: html,
      页面设置: (项.type === 'word' || 项.type === undefined) && 文字保存快照 !== undefined && 项.页面设置 ===
        ? 文字保存快照.页面设置 : 项.页面设置,
      已保存页面设置: 项.type === 'word' || 项.type === undefined
        ? (文字保存快照 === undefined ? 项.页面设置 : 文字保存快照.页面设置)
        : 项.已保存页面设置,
      已保存模型: 项.type === 'word' || 项.type === undefined ? 项.已保存模型 : (模型快照 ?? JSON.stringify(模型
    }
  }))
}

const set文档路径 = (标识: string, 路径: string | null) => {
  set文档路径状态((当前) => ({ ...当前, [标识]: 路径 }))
  if (路径 !== null) {
    setDocuments((当前) => 当前.map((文档) => 文档.id === 标识 ? { ...文档, name: 基准文件名(路径) } : 文档))
  }
}
```

```
const 更新文件指纹 = (标识: string, 指纹: string) => {
  setDocuments((当前) => 当前.map((文档) => 文档.id === 标识 ? { ...文档, 文件指纹: 指纹 } : 文档))
}

const 查找保存路径占用 = (当前标识: string | null, 目标路径: string): WorkspaceTab | null =>
  workspaceTabsRef.current.find((标签) => 标签.id !== 当前标识 && 是同一路径(标签.path, 目标路径)) ?? null

const 选择编辑文档 = (标识: string | null) => {
  if (标识 === null) {
    setActiveDocumentId(null)
    return
  }
  const 文档 = documents.find((项) => 项.id === 标识)
  if (!文档) throw new Error('找不到要切换的编辑文档')
  setActiveDocumentId(标识)
  setModule(文档.type ?? 'word')
}

const closeEditorDoc = (标识: string) => {
  const 原位置 = workspaceOrder.indexOf(标识)
  const 剩余顺序 = workspaceOrder.filter((项) => 项 !== 标识)
  const 下个标识 = 剩余顺序[Math.min(Math.max(原位置 - 1, 0), 剩余顺序.length - 1)]
  const 正在显示 = activeWorkspaceTabId === 标识
  const 剩余 = documents.filter((项) => 项.id !== 标识)
  setDocuments(剩余)
  setWorkspaceOrder(剩余顺序)
  set表格文档模型((当前) => { const 结果 = { ...当前 }; delete 结果[标识]; return 结果 })
  set演示文档模型((当前) => { const 结果 = { ...当前 }; delete 结果[标识]; return 结果 })
  设置演示历史资源(标识, [])
  set文档路径状态((当前) => { const 结果 = { ...当前 }; delete 结果[标识]; return 结果 })
  if (!正在显示) {
    if (activeDocumentId === 标识) setActiveDocumentId(剩余[剩余.length - 1]?.id ?? null)
    return
  }
  if (下个标识 === undefined) {
    setActiveDocumentId(null)
    setActiveDocId(null)
    setModule('home')
    return
  }
  selectWorkspaceTab(下个标识)
}

const selectWorkspaceTab = (标识: string) => {
  if (标识 === 'home') {
    goHome()
    return
```

```
  }
  const 文档 = documents.find((项) => 项.id === 标识)
  if (文档) {
    选择编辑文档(标识)
    setNavKey(文档.type ?? 'word')
    return
  }
  const PDF文档 = pdfDocuments.find((项) => 项.id === 标识)
  if (PDF文档) {
    setActivePdfId(标识)
    setPDF待预览(预览PDF文档(PDF文档))
    setModule('pdf')
    setNavKey('pdf')
    return
  }
  throw new Error('找不到要切换的工作区标签')
}

const 更新PDF内容 = (标识:string, 数据:string) => {
  setPdfDocuments(当前 => 当前.map(文档 => 文档.id === 标识 ? {...文档,data:数据,dirty:true} : 文档))
  if (activePdfId === 标识) setPDF待预览(当前 => 当前 ? {...当前,数据} : 当前)
}
const 保存PDF文档 = (标识:string, 路径:string, 数据:string) => {
  const 文档 = pdfDocuments.find(d => d.id === 标识)
  if (!文档) throw new Error('PDF 标签已关闭')
  if (pdfDocuments.some(d => d.id !== 标识 && 是同一路径(d.path, 路径))) throw new Error('此路径已在其他 PDF
  setPdfDocuments(当前 => 当前.map(d => d.id === 标识 ? {...d,path:路径,name:基准文件名(路径),data:数据,dirty:
  if (activePdfId === 标识) setPDF待预览({路径,名称:基准文件名(路径),数据})
}

const closeWorkspaceTab = (标识: string) => {
  if (标识 === 'home') return
  if (documents.some((项) => 项.id === 标识)) {
    closeEditorDoc(标识)
    return
  }
  if (!pdfDocuments.some((项) => 项.id === 标识)) return
  const 原位置 = workspaceOrder.indexOf(标识)
  const 剩余顺序 = workspaceOrder.filter((项) => 项 !== 标识)
  const 下个标识 = 剩余顺序[Math.min(Math.max(原位置 - 1, 0), 剩余顺序.length - 1)]
  setPdfDocuments((当前) => 当前.filter((项) => 项.id !== 标识))
  setWorkspaceOrder(剩余顺序)
  if (activeWorkspaceTabId === 标识) {
    if (下个标识) selectWorkspaceTab(下个标识)
    else {
      setActivePdfId(null)
      setPDF待预览(null)
      goHome()
```

（中间部分省略）

```
  const 历史 = 读取修订(元素)
  if (历史) 新格式.修订 = [...(父格式.修订 ?? []), 历史]
  return 新格式
}

function 读取修订(元素: HTMLElement): 原生修订记录 | undefined {
  const 类型 = 元素.dataset.sealRevisionType
  if (!['ins', 'del', 'moveFrom', 'moveTo'].includes(类型 ?? '')) return undefined
  const 标识 = 元素.dataset.sealRevisionId ?? '', 作者 = 元素.dataset.sealRevisionAuthor ?? '', 时间 = 元素
  if (!/^\d{1,10}$/.test(标识) || Number(标识) > 2147483647 || 作者.length > 512 || 时间 !== undefined
  return { 类型: 类型 as 原生修订记录['类型'], 标识, 作者, ...(时间 ? { 时间 } : {}) }
}

function 读取字体尺寸(值: string, 父字号 = 10.5): number | undefined {
  const 相对 = 值.trim().match(/^(\d+(?:\.\d+)?)(em|%)$/)
  return 相对 ? Number(相对[1]) * 父字号 / (相对[2] === '%' ? 100 : 1) : 长度转磅(值)
}

/** 把格式状态与文本合成一个片段 */
function 建片段(文本: string, 格式: 格式状态): 文字片段 {
  return {
    文本,
    ...(格式.修订?.length ? { 修订: 格式.修订 } : {}),
    ...(格式.格式修订 ? { 格式修订: 格式.格式修订 } : {}),
    加粗: 格式.加粗,
    倾斜: 格式.倾斜,
    下划线: 格式.下划线,
    删除线: 格式.删除线,
    颜色: 格式.颜色,
    底纹: 格式.底纹,
    字号: 格式.字号,
    字体: 格式.字体,
    ...(格式.字间距 !== undefined ? { 字间距: 格式.字间距 } : {}),
    ...(格式.基线 ? { 基线: 格式.基线 } : {}),
    ...(格式.链接 ? { 链接: 格式.链接 } : {}),
  }
}

/** 页码类域在编辑区里的标记属性，与主进程读取端保持一致 */
const 域标记属性 = 'data-seal-field'
const 支持的域 = new Set(['PAGE', 'NUMPAGES'])

/**
 * 读取页码域占位元素；返回 null 表示该元素不是域标记。
 * 未知域无法写回，记入未覆盖清单以便保存前如实告知。
 */
function 读取域片段(元素: HTMLElement, 格式: 格式状态, 未覆盖: Set<string>): 文字片段 | null {
  const 域名称 = (元素.getAttribute(域标记属性) ?? '').toUpperCase()
```

```
  if (域名称 === '') {
    return null
  }
  if (!支持的域.has(域名称)) {
    未覆盖.add('域代码')
    return null
  }
  const 片段格式 = 叠加格式(元素, 格式)
  return { ...建片段(元素.textContent ?? '', 片段格式), 域: 域名称 as 'PAGE' | 'NUMPAGES' }
}

/** 解析过程中的可变上下文 */
interface 解析上下文 {
  段落: 段落模型[]
  未覆盖: Set<string>
  /** 当前正在收集片段的文本段落，遇到块级边界时结算 */
  当前: 文本段落 | null
}

/** 取得当前段落，没有则按给定属性新建一个 */
function 取当前段落(上下文: 解析上下文, 级别: number, 对齐: 对齐方式, 列表: 列表类型): 文本段落 {
  if (上下文.当前 === null) {
    上下文.当前 = { 类型: '段落', 级别, 对齐, 列表, 文字: [] }
    上下文.段落.push(上下文.当前)
  }
  return 上下文.当前
}

/** 结束当前段落的收集，下一段文字会开启新段落 */
function 结算段落(上下文: 解析上下文): void {
  上下文.当前 = null
}

/** 读取元素的对齐方式，兼容内联样式与旧式 align 属性 */
function 读对齐(元素: HTMLElement): 对齐方式 | null {
  const 取值 = (元素.style.textAlign || 元素.getAttribute('align') || '').toLowerCase()
  switch (取值) {
    case 'center':
      return '中'
    case 'right':
      return '右'
    case 'justify':
      return '两端'
    case 'left':
      return '左'
    default:
      return null
  }
```

```
}

/** 判断元素是否位于某种列表中，返回列表类型 */
function 读列表(元素: HTMLElement): 列表类型 {
  const 父 = 元素.parentElement
  if (父 === null) {
    return '无'
  }
  if (父.tagName === 'OL') {
    return '编号'
  }
  if (父.tagName === 'UL') {
    return '项目符号'
  }
  return '无'
}

/** 把节点内的全部文字收集为片段数组，忽略其中的块级结构 */
function 收集片段(节点: Node, 格式: 格式状态, 未覆盖: Set<string>): 文字片段[] {
  const 结果: 文字片段[] = []

  const 遍历 = (当前: Node, 当前格式: 格式状态): void => {
    if (当前.nodeType === Node.TEXT_NODE) {
      const 文本 = 当前.textContent ?? ''
      if (文本.length > 0) {
        结果.push(建片段(文本, 当前格式))
      }
      return
    }
    if (!(当前 instanceof HTMLElement)) {
      return
    }
    if (当前.tagName === 'IMG') {
      const 图片 = 读取文档图片(当前 as HTMLImageElement, 未覆盖)
      if (图片) 结果.push({ ...建片段('', 当前格式), 图片 })
      return
    }
    if (当前.tagName === 'BR') {
      结果.push({ ...建片段('', 当前格式), 换行: true })
      return
    }
    if (当前.hasAttribute('data-seal-bookmark')) {
      const 书签 = 读取书签名称(当前, 未覆盖)
      if (书签 !== undefined) 结果.push({ ...建片段('', 当前格式), 书签 })
      return
    }
    // 页码域占位：保留域类型与它上次显示的文字，保存时写回真正的域代码
    const 域片段 = 读取域片段(当前, 当前格式, 未覆盖)
```

```
    if (域片段 !== null) {
      结果.push(域片段)
      return
    }
    const 子格式 = 叠加格式(当前, 当前格式)
    当前.childNodes.forEach((子) => 遍历(子, 子格式))
  }

  节点.childNodes.forEach((子) => 遍历(子, 格式))
  return 结果
}

/** 读取合并跨度：只接受 1 至 256，越界或非法取值按未覆盖如实报告 */
function 读取合并跨度(取值: string | null, 未覆盖: Set<string>): number {
  if (取值 === null) return 1
  const 数字 = Number(取值)
  if (!/^\d+$/.test(取值) || !Number.isSafeInteger(数字) || 数字 < 1 || 数字 > 256) {
    未覆盖.add('合并单元格')
    return 1
  }
  return 数字
}

/** 解析一个表格元素，行列为空时返回 null */
function 解析表格(表: HTMLTableElement, 未覆盖: Set<string>): 表格段落 | null {
  const 行列表 = Array.from(表.querySelectorAll('tr'))
  const 行: 表格单元[][] = []

  行列表.forEach((行元素) => {
    const 单元列表 = Array.from(行元素.querySelectorAll('th, td'))
    if (单元列表.length === 0) {
      return
    }
    行.push(
      单元列表.map((单元) => {
        const 元素 = 单元 as HTMLElement
        // 被合并覆盖的单元格不会出现在后续行列里，跨度写入模型后由保存端还原
        const 跨列 = 读取合并跨度(元素.getAttribute('colspan'), 未覆盖)
        const 跨行 = 读取合并跨度(元素.getAttribute('rowspan'), 未覆盖)
        const 单元上下文: 解析上下文 = { 段落: [], 当前: null, 未覆盖 }
        const 单元格式 = 叠加格式(元素, 初始格式)
        元素.childNodes.forEach((子) => 遍历节点(子, 单元格式, 单元上下文))
        if (单元上下文.段落.some((段) => 段.类型 !== '段落')) 未覆盖.add('表格内嵌对象')
        return {
          表头: 元素.tagName === 'TH',
          ...(跨列 > 1 ? { 跨列 } : {}),
          ...(跨行 > 1 ? { 跨行 } : {}),
          文字: 收集片段(元素, 单元格式, 未覆盖),
```

```
          段落: 单元上下文.段落.filter((段): 段 is 文本段落 => 段.类型 === '段落'),
          宽度: 长度转磅(元素.style.width),
          内边距: { 上: 长度转磅(元素.style.paddingTop) ?? 0, 右: 长度转磅(元素.style.paddingRight) ?? 0,
            下: 长度转磅(元素.style.paddingBottom) ?? 0, 左: 长度转磅(元素.style.paddingLeft) ?? 0 },
          边框: Object.fromEntries(['top', 'right', 'bottom', 'left'].map(side => [side, {
            样式: 元素.style.getPropertyValue(`border-${side}-style`) || 'none',
            宽: 长度转磅(元素.style.getPropertyValue(`border-${side}-width`)) ?? 0,
            颜色: 颜色转十六进制(元素.style.getPropertyValue(`border-${side}-color`)),
          }])),
          底纹: 颜色转十六进制(元素.style.backgroundColor),
          垂直对齐: 元素.style.verticalAlign || undefined,
        }
      })
    )
  })

  return 行.length > 0 ? { 类型: '表格', 行, 宽度: 长度转磅(表.style.width),
    ...(表.style.width.endsWith('%') ? { 宽度百分比: Number.parseFloat(表.style.width) } : {}),
    边框合并: 表.style.borderCollapse === 'collapse', 单元间距: 长度转磅(表.style.borderSpacing.split(' ')[0]
}

/** 显式包裹空格的行内标签：其中的空格是真实内容，不当作缩进空白丢弃 */
const 行内标签 = new Set(['SPAN', 'FONT', 'B', 'I', 'U', 'S', 'STRIKE', 'EM', 'STRONG', 'SUP', 'SUB

/** 遍历 DOM 节点，把内容填入解析上下文 */
function 遍历节点(节点: Node, 格式: 格式状态, 上下文: 解析上下文): void {
  if (节点.nodeType === Node.TEXT_NODE) {
    const 文本 = 节点.textContent ?? ''
    if (文本.length === 0) {
      return
    }
    // 只有空格的文本节点若是被行内标签显式包裹（如 <span> </span>），就是正文里的真实空格；
    // 其余不带内容的空白来自 HTML 缩进，不构成内容
    if (文本.trim().length === 0 && !(节点.parentElement !== null && 行内标签.has(节点.parentElement.tagN
      return
    }
    取当前段落(上下文, 0, '左', '无').文字.push(建片段(文本, 格式))
    return
  }

  if (!(节点 instanceof HTMLElement)) {
    return
  }

  const 标签 = 节点.tagName
  if (节点.hasAttribute('data-seal-docx-package')) return
  if (节点.hasAttribute('data-seal-ooxml-block')) {
    结算段落(上下文)
```

```
  const start = 上下文.段落.length
  节点.childNodes.forEach(child => 遍历节点(child, 叠加格式(节点, 格式), 上下文))
  if (节点.dataset.sealOoxmlEmpty === 'true' && 上下文.段落.length === start) 上下文.段落.push({ 类型: '段落'
  for (const block of 上下文.段落.slice(start)) block.原始块 = { 标识: 节点.dataset.sealOoxmlBlock!, 未修改:
  结算段落(上下文)
  return
}

if (节点.hasAttribute('data-seal-note-ref')) {
  const kind = 节点.dataset.sealNoteKind, id = Number(节点.dataset.sealNoteRef)
  if ((kind === 'footnote' || kind === 'endnote') && Number.isSafeInteger(id) && id > 0 && id
    取当前段落(上下文, 0, '左', '无').文字.push({ ...建片段('', 格式), 注释引用: { 类型: kind, 标识: id } })
  } else 上下文.未覆盖.add('无效注释引用')
  return
}

if (标签 === 'HR') {
  结算段落(上下文)
  上下文.段落.push({ 类型: '水平线' })
  return
}

if (标签 === 'BR') {
  // 空段落的占位 br 不表示额外换行；其他 br 保留为段内软回车。
  const 是空行占位 = 节点.parentElement && 块级标签.has(节点.parentElement.tagName) &&
    Array.from(节点.parentElement.childNodes).every((子) => 子 === 节点 || 子.nodeType === Node.TEXT
  const 段 = 取当前段落(上下文, 0, '左', '无')
  if (!是空行占位) 段.文字.push({ ...建片段('', 格式), 换行: true })
  return
}

if (标签 === 'IMG') {
  const 图片 = 读取文档图片(节点 as HTMLImageElement, 上下文.未覆盖)
  if (图片) 取当前段落(上下文, 0, '左', '无').文字.push({ ...建片段('', 格式), 图片 })
  return
}

if (节点.hasAttribute('data-seal-bookmark')) {
  const 书签 = 读取书签名称(节点, 上下文.未覆盖)
  if (书签 !== undefined) 取当前段落(上下文, 0, '左', '无').文字.push({ ...建片段('', 格式), 书签 })
  return
}

// 页码域占位：正文与页脚里的页码都不会被固定成保存时看到的数字
const 域片段 = 读取域片段(节点, 格式, 上下文.未覆盖)
if (域片段 !== null) {
  取当前段落(上下文, 0, '左', '无').文字.push(域片段)
  return
```

```
}

if (节点.classList.contains('wps-page-break')) {
  结算段落(上下文)
  上下文.段落.push({ 类型: '分页符' })
  return
}

if (标签 === 'TABLE') {
  结算段落(上下文)
  const 表 = 解析表格(节点 as HTMLTableElement, 上下文.未覆盖)
  if (表 !== null) {
    上下文.段落.push(表)
  }
  return
}

// 浏览器缩进和粘贴产生的外层容器没有独立正文；将其布局应用到实际段落。
const 是段落容器 = 块级标签.has(标签) || 标签 === 'UL' || 标签 === 'OL'
const 子节点 = Array.from(节点.childNodes)
const 只含块级内容 = 子节点.some((子) => 子 instanceof HTMLElement) && 子节点.every((子) =>
  子.nodeType === Node.TEXT_NODE ? !子.textContent?.trim() : 子 instanceof HTMLElement &&
    (块级标签.has(子.tagName) || ['UL', 'OL', 'TABLE'].includes(子.tagName)))
if (是段落容器 && 只含块级内容) {
  结算段落(上下文)
  const 子格式 = 叠加格式(节点, 格式)
  const 排版 = 读取段落排版(节点, 子格式.字号, 上下文.未覆盖)
  const 起点 = 上下文.段落.length
  子节点.forEach((子) => 遍历节点(子, 子格式, 上下文))
  const 段落 = 上下文.段落.slice(起点).filter((段): 段 is 文本段落 => 段.类型 === '段落')
  if (排版.缩进 && 段落.length !== 上下文.段落.length - 起点) 上下文.未覆盖.add('表格外层容器排版')
  段落.forEach((段) => {
    if (排版.缩进) {
      段.缩进 ??= {}
      for (const 字段 of ['左', '右'] as const) {
        if (排版.缩进[字段] !== undefined) 段.缩进[字段] = (段.缩进[字段] ?? 0) + 排版.缩进[字段]!
      }
      if (段.缩进.首行 === undefined && 段.缩进.悬挂 === undefined) {
        if (排版.缩进.首行 !== undefined) 段.缩进.首行 = 排版.缩进.首行
        if (排版.缩进.悬挂 !== undefined) 段.缩进.悬挂 = 排版.缩进.悬挂
      }
    }
    if (排版.间距?.行距 !== undefined && 段.间距?.行距 === undefined) {
      段.间距 = { ...段.间距, 行距: 排版.间距.行距, 行距规则: 排版.间距.行距规则 }
    }
  })
  if (段落.length && 排版.间距) {
    for (const [段, 字段] of [[段落[0], '段前'], [段落[段落.length - 1], '段后']] as const) {
```

```
      if (排版.间距[字段] !== undefined) 段.间距 = { ...段.间距, [字段]: Math.max(段.间距?.[字段] ?? 0, 排版.间距[字段
    }
  }
  结算段落(上下文)
  return
}

if (标签 === 'UL' || 标签 === 'OL' || 标签 === 'TBODY' || 标签 === 'THEAD') {
  结算段落(上下文)
  节点.childNodes.forEach((子) => 遍历节点(子, 格式, 上下文))
  结算段落(上下文)
  return
}

if (块级标签.has(标签)) {
  结算段落(上下文)
  const 级别 = 标题级别[标签] ?? 0
  const 对齐 = 读对齐(节点) ?? '左'
  const 列表 = 标签 === 'LI' ? 读列表(节点) : '无'
  const 段 = 取当前段落(上下文, 级别, 对齐, 列表)
  if (列表 !== '无') {
    const style = 节点.style.listStyleType || (节点.closest('ol,ul') as HTMLElement | null)?.styl
    if (style && ['decimal', 'cjk-ideographic', 'lower-alpha', 'disc', 'square', 'circle'].in
  }
  const 子格式 = 叠加格式(节点, 格式)
  Object.assign(段, 读取段落排版(节点, 子格式.字号, 上下文.未覆盖))
  段.标记 = 建片段('', 子格式)
  if (节点.dataset.sealParagraphRevision) {
    try { const revision = JSON.parse(decodeURIComponent(节点.dataset.sealParagraphRevision));
  }

  节点.childNodes.forEach((子) => {
    // 块级元素内若再嵌套块级元素，由递归自行结算
    遍历节点(子, 子格式, 上下文)
  })

  // 段落内没有产生任何文字时仍保留，作为空行
  if (上下文.当前 === 段 || 段.文字.length === 0) {
    结算段落(上下文)
  } else {
    结算段落(上下文)
  }
  return
}

// 行内元素：格式向下继承，文字并入当前段落
const 子格式 = 叠加格式(节点, 格式)
节点.childNodes.forEach((子) => 遍历节点(子, 子格式, 上下文))
```

```
}

/**
 * 把编辑区 HTML 解析为文档模型。
 * @param 正文Html - 编辑区的 innerHTML
 */
export function 解析文档(正文Html: string, 部分: '正文' | '页眉' | '页脚' = '正文', 设置?: 文字页面设置): 文档模型 {
  const 容器 = document.createElement('div')
  容器.innerHTML = 正文Html
  const source = 容器.querySelector<HTMLElement>('[data-seal-docx-package]'), 源包 = source?.datase
  if (源包 && (源包.length > 28_000_000 || !/^[A-Za-z0-9+/]+={0,2}$/.test(源包))) throw new Error('DO
  let 修订动作: 文档模型['修订动作']
  let 源包基准: 文档模型['源包基准']
  if (source?.dataset.sealDocxBaseline) { try { 源包基准 = JSON.parse(decodeURIComponent(source.dat
  if (source?.dataset.sealRevisionActions) { try { 修订动作 = JSON.parse(decodeURIComponent(source.
  source?.remove()
  const canonicalHtml = (html: string) => { const box = document.createElement('div'); box.inne
  const sourceIds = new Set<string>()
  for (const node of 容器.querySelectorAll<HTMLElement>('[data-seal-ooxml-block]')) {
    if (!/^b\d{1,5}$/.test(node.dataset.sealOoxmlBlock ?? '') || sourceIds.has(node.dataset.sea
    sourceIds.add(node.dataset.sealOoxmlBlock!)
    let baseline = ''
    try { baseline = decodeURIComponent(node.dataset.sealOoxmlBaseline ?? '') } catch { throw n
    node.dataset.sealOoxmlUnchanged = String(canonicalHtml(node.innerHTML) === canonicalHtml(ba
  }
  const 注释: NonNullable<文档模型['注释']> = [], 注释警告 = new Set<string>()
  for (const section of [...容器.querySelectorAll<HTMLElement>('[data-seal-notes]')]) {
    const kind = section.dataset.sealNotes
    for (const definition of [...section.querySelectorAll<HTMLElement>('[data-seal-note-id]')])
      const id = Number(definition.dataset.sealNoteId)
      if (!['footnote', 'endnote'].includes(kind || '') || !Number.isSafeInteger(id) || id <= 0
      definition.querySelectorAll('[data-seal-note-label]').forEach(label => label.remove())
      const parsed = 解析文档(definition.innerHTML, 部分, 设置)
      parsed.未覆盖.forEach(warning => 注释警告.add(`注释：${warning}`))
      let baseline = ''
      try { baseline = decodeURIComponent(definition.dataset.sealNoteBaseline ?? '') } catch {
      注释.push({ 类型: kind as 'footnote' | 'endnote', 标识: id, 段落: parsed.段落, 未修改: !!baseline && c
    }
    section.remove()
  }
  固定文档显示格式(容器, 部分, 设置)

  const 上下文: 解析上下文 = { 段落: [], 未覆盖: new Set<string>(), 当前: null }
  for (const node of 容器.querySelectorAll<HTMLElement>('[data-seal-format-change]')) { try { con
  if (容器.querySelector('.wps-chart')) 上下文.未覆盖.add('图表')
  if (容器.querySelector('.wps-smartart')) 上下文.未覆盖.add('智能图形')
  if (容器.querySelector('.wps-toc')) 上下文.未覆盖.add('自动目录')
  if (容器.querySelector('.wps-drop-cap')) 上下文.未覆盖.add('首字下沉')
```

```
  let 新修订号 = Math.max(0, ...Array.from(容器.querySelectorAll<HTMLElement>('[data-seal-revision-id
  for (const node of 容器.querySelectorAll<HTMLElement>('.wps-insert,.wps-delete,ins,del,[data-se
    if (!node.dataset.sealRevisionType) { node.dataset.sealRevisionType = node.classList.contai
    if (!读取修订(node)) 上下文.未覆盖.add('修订标记无效')
  }
  for (const 链接 of 容器.querySelectorAll<HTMLElement>('a')) {
    const 有目标 = 链接.hasAttribute('data-seal-link') || 链接.hasAttribute('href')
    if (有目标 && 读取链接目标(链接) === undefined) 上下文.未覆盖.add('超链接目标')
  }
  if (容器.querySelector('svg')) 上下文.未覆盖.add('矢量图形')
  if (容器.querySelector('video, audio, canvas, iframe, object, embed')) 上下文.未覆盖.add('媒体或嵌入对象')
  for (const 元素 of 容器.querySelectorAll<HTMLElement>('p, div, h1, h2, h3, h4, h5, h6, li, blockq
    const 样式 = 元素.style
    const 有实际尺寸 = (值: string) => 值 !== '' && !/^(?:0(?:px|pt|em|rem|cm|mm|in|pc)?\s*)+$/.test(值
    if (样式.border && 样式.border !== 'none' && (!样式.borderWidth || 有实际尺寸(样式.borderWidth)) || 有实际尺
        有实际尺寸(样式.padding) || 有实际尺寸(样式.paddingTop) || 有实际尺寸(样式.paddingBottom) ||
        样式.boxShadow && 样式.boxShadow !== 'none' || 样式.transform && 样式.transform !== 'none' ||
        样式.position && 样式.position !== 'static' || 样式.float && 样式.float !== 'none' ||
        (样式.width && 元素.tagName === 'DIV')) {
      上下文.未覆盖.add('图形或文本框布局')
    }
  }
  const 根格式 = 叠加格式(容器, 初始格式)
  容器.childNodes.forEach((子) => 遍历节点(子, 根格式, 上下文))

  // 空文档也要给出一个空段落，否则生成的文件结构非法
  if (上下文.段落.length === 0) {
    上下文.段落.push({ 类型: '段落', 级别: 0, 对齐: '左', 列表: '无', 文字: [] })
  }

  return { 段落: 上下文.段落, ...(源包 ? { 源包, 源包基准 } : {}), ...(修订动作?.length ? { 修订动作 } : {}), ...(注释.l
}

// ============ FILE: shortcuts\commandCatalog.ts ============
// Generated by scripts/generate-shortcut-catalog.cjs from implemented command registries.
export const commandCatalog = {
  "word": [
    [
      "bibliography.insert",
      "书目"
    ],
    [
      "bookmark.insert",
      "书签"
    ],
    [
      "caption.insert",
      "插入题注"
```

```
],
[
  "caption.tableOfFigures",
  "插入表目录"
],
[
  "chart.insert",
  "图表"
],
[
  "citation.insert",
  "插入引文"
],
[
  "citation.manageSource",
  "管理源"
],
[
  "clipboard.copy",
  "复制"
],
[
  "clipboard.cut",
  "剪切"
],
[
  "clipboard.formatPainter",
  "格式刷"
],
[
  "clipboard.paste",
  "粘贴"
],
[
  "comment.delete",
  "删除批注"
],
[
  "comment.new",
  "新建批注"
],
[
  "comment.show",
  "显示批注"
],
[
  "compare.start",
  "比较"
```

```
],
[
  "crossref.insert",
  "交叉引用"
],
[
  "datetime.insert",
  "日期时间"
],
[
  "edit.find",
  "查找"
],
[
  "edit.redo",
  "重做"
],
[
  "edit.replace",
  "替换"
],
[
  "edit.selectAll",
  "全选"
],
[
  "edit.undo",
  "撤销"
],
[
  "endnote.insert",
  "插入尾注"
],
[
  "field.insert",
  "文档部件"
],
[
  "file.exportHtml",
  "导出为网页"
],
[
  "file.exportPdf",
  "导出为PDF"
],
[
  "file.exportText",
  "导出为文本"
```

```
  ],
  [
    "file.open",
    "打开文件"
  ],
  [
    "file.save",
    "保存"
  ],
  [
    "file.saveAs",
    "另存为"
  ],
  [
    "font.bold",
    "加粗"
  ],
  [
    "font.clear",
    "清除格式"
  ],
  [
    "font.color",
    "字体颜色"
  ],
  [
    "font.grow",
    "增大字号"
  ],
  [
    "font.highlight",
    "突出显示"
  ],
  [
    "font.italic",
    "斜体"
  ],
  [
    "font.name",
    "字体"
  ],
  [
    "font.shrink",
    "减小字号"
  ],
  [
    "font.size",
    "字号"
```

```
],
[
  "font.strike",
  "删除线"
],
[
  "font.subscript",
  "下标"
],
[
  "font.superscript",
  "上标"
],
[
  "font.underline",
  "下划线"
],
[
  "footer.edit",
  "页脚"
],
[
  "footnote.insert",
  "插入脚注"
],
[
  "formula.insert",
  "公式"
],
[
  "header.edit",
  "页眉"
],
[
  "image.insert",
  "图片"
],
[
  "lang.convert",
  "简繁转换"
],
[
  "layout.break",
  "分隔符"
],
[
  "layout.columns",
  "分栏"
```

```
],
[
  "layout.dropCap",
  "首字下沉"
],
[
  "layout.lineNumbers",
  "行号"
],
[
  "layout.margin",
  "页边距"
],
[
  "layout.orientation",
  "纸张方向"
],
[
  "layout.pageBorder",
  "页面边框"
],
[
  "layout.pageColor",
  "页面颜色"
],
[
  "layout.paperSize",
  "纸张大小"
],
[
  "layout.textDirection",
  "文字方向"
],
[
  "layout.watermark",
  "水印"
],
[
  "link.insert",
  "超链接"
],
[
  "mailmerge.start",
  "邮件合并"
],
[
  "merge.start",
  "合并"
```

```
],
[
  "page.blank",
  "空白页"
],
[
  "page.break",
  "分页符"
],
[
  "page.cover",
  "封面"
],
[
  "pagenumber.insert",
  "页码"
],
[
  "para.alignCenter",
  "居中"
],
[
  "para.alignJustify",
  "两端对齐"
],
[
  "para.alignLeft",
  "左对齐"
],
[
  "para.alignRight",
  "右对齐"
],
[
  "para.bullet",
  "项目符号"
],
[
  "para.indentDecrease",
  "减少缩进"
],
[
  "para.indentIncrease",
  "增加缩进"
],
[
  "para.lineSpacing",
  "行距"
```

```
],
[
  "para.number",
  "编号"
],
[
  "para.shading",
  "段落底纹"
],
[
  "protect.start",
  "保护文档"
],
[
  "shape.insert",
  "形状"
],
[
  "smartart.insert",
  "SmartArt"
],
[
  "spell.check",
  "拼写检查"
],
[
  "style.body",
  "正文"
],
[
  "style.heading1",
  "标题 1"
],
[
  "style.heading2",
  "标题 2"
],
[
  "style.heading3",
  "标题 3"
],
[
  "symbol.insert",
  "符号"
],
[
  "table.draw",
  "绘制表格"
```

```
],
[
  "table.insert",
  "插入表格"
],
[
  "textbox.insert",
  "文本框"
],
[
  "toc.insert",
  "目录"
],
[
  "toc.update",
  "更新目录"
],
[
  "track.accept",
  "接受修订"
],
[
  "track.enable",
  "修订"
],
[
  "track.history",
  "修订历史"
],
[
  "track.reject",
  "拒绝修订"
],
[
  "translate.start",
  "翻译"
],
[
  "view.draft",
  "草稿"
],
[
  "view.gridlines",
  "网格线"
],
[
  "view.navigation",
  "导航窗格"
```

```
],
[
  "view.outline",
  "大纲视图"
],
[
  "view.page",
  "页面视图"
],
[
  "view.paragraphMark",
  "显示段落标记"
],
[
  "view.read",
  "阅读版式"
],
[
  "view.ruler",
  "标尺"
],
[
  "view.web",
  "Web 版式"
],
[
  "view.zoomFitWidth",
  "适应页宽"
],
[
  "view.zoomIn",
  "放大"
],
[
  "view.zoomOut",
  "缩小"
],
[
  "view.zoomReset",
  "100%"
],
[
  "word.count",
  "字数统计"
],
[
  "wordart.insert",
  "艺术字"
```

```
    ]
  ],
  "sheet": [
    [
      "cell.alignCenter",
      "居中"
    ],
    [
      "cell.alignLeft",
      "左对齐"
    ],
    [
      "cell.alignRight",
      "右对齐"
    ],
    [
      "cell.bold",
      "加粗"
    ],
    [
      "cell.border",
      "边框"
    ],
    [
      "cell.delete",
      "删除单元格"
    ],
    [
      "cell.fill",
      "填充颜色"
    ],
    [
      "cell.fontColor",
      "字体颜色"
    ],
    [
      "cell.insert",
      "插入单元格"
    ],
    [
      "cell.italic",
      "斜体"
    ],
    [
      "cell.mergeCenter",
      "合并后居中"
    ],
    [
```

```
  "cell.underline",
  "下划线"
],
[
  "cell.wrap",
  "自动换行"
],
[
  "clipboard.copy",
  "复制"
],
[
  "clipboard.cut",
  "剪切"
],
[
  "clipboard.paste",
  "粘贴"
],
[
  "col.delete",
  "删除列"
],
[
  "col.insert",
  "插入列"
],
[
  "data.filter",
  "筛选"
],
[
  "data.removeDuplicates",
  "删除重复项"
],
[
  "data.sortAsc",
  "升序"
],
[
  "data.sortDesc",
  "降序"
],
[
  "data.textToColumns",
  "分列"
],
[
```

```
  "data.validation",
  "数据验证"
],
[
  "edit.clear",
  "清除内容"
],
[
  "edit.find",
  "查找"
],
[
  "edit.redo",
  "重做"
],
[
  "edit.selectAll",
  "全选"
],
[
  "edit.sum",
  "自动求和"
],
[
  "edit.undo",
  "撤销"
],
[
  "formula.financial",
  "财务函数"
],
[
  "formula.logical",
  "逻辑函数"
],
[
  "formula.lookup",
  "查找与引用"
],
[
  "insert.chart",
  "图表"
],
[
  "insert.picture",
  "图片"
],
[
```

```
  "layout.margin",
  "页边距"
],
[
  "layout.orientation",
  "纸张方向"
],
[
  "layout.paperSize",
  "纸张大小"
],
[
  "number.currency",
  "货币"
],
[
  "number.format",
  "数字格式"
],
[
  "number.percent",
  "百分比"
],
[
  "number.plain",
  "常规"
],
[
  "number.thousand",
  "千位分隔"
],
[
  "review.comment",
  "批注"
],
[
  "review.protect",
  "保护工作表"
],
[
  "row.delete",
  "删除行"
],
[
  "row.insert",
  "插入行"
],
[
```

```
      "spell.check",
      "拼写检查"
    ],
    [
      "symbol.insert",
      "插入符号"
    ],
    [
      "view.freeze",
      "冻结窗格"
    ],
    [
      "view.normal",
      "普通视图"
    ],
    [
      "view.pageLayout",
      "页面布局视图"
    ],
    [
      "view.split",
      "拆分"
    ]
  ],
  "ppt": [
    [
      "animation.appear",
      "出现"
    ],
    [
      "animation.fade",
      "淡出"
    ],
    [
      "box.new",
      "文本框"
    ],
    [
      "clipboard.copy",
      "复制"
    ],
    [
      "clipboard.paste",
      "粘贴"
    ],
    [
      "design.background",
      "背景色"
```

```
],
[
  "design.background.all",
  "背景应用到全部"
],
[
  "design.background.clear",
  "清除背景"
],
[
  "design.background.gradient",
  "渐变背景"
],
[
  "design.background.image",
  "图片背景"
],
[
  "design.beautify.all",
  "本机美化全部"
],
[
  "design.beautify.page",
  "本机美化当前页"
],
[
  "design.beautify.preview",
  "预览本机美化"
],
[
  "design.check",
  "检查与美化"
],
[
  "design.color.apply",
  "配色方案"
],
[
  "design.font.body",
  "正文字体"
],
[
  "design.font.title",
  "标题字体"
],
[
  "design.footer.date",
  "日期"
```

```
],
[
  "design.footer.firstPageOff",
  "首页不显示"
],
[
  "design.footer.pageNumber",
  "页码"
],
[
  "design.footer.text",
  "页脚"
],
[
  "design.inherit.release",
  "解除继承"
],
[
  "design.inherit.restore",
  "恢复继承"
],
[
  "design.layout.apply",
  "应用版式"
],
[
  "design.layout.create",
  "新增版式"
],
[
  "design.master",
  "幻灯片母版"
],
[
  "design.master.sync",
  "同步占位符"
],
[
  "design.size.16:9",
  "宽屏 16:9"
],
[
  "design.size.4:3",
  "标准 4:3"
],
[
  "design.size.custom",
  "自定义尺寸"
```

```
],
[
  "design.size.orientation",
  "方向"
],
[
  "design.theme",
  "主题"
],
[
  "design.theme.apply",
  "应用主题"
],
[
  "design.theme.preview",
  "预览主题"
],
[
  "edit.redo",
  "重做"
],
[
  "edit.undo",
  "撤销"
],
[
  "insert.chart",
  "图表"
],
[
  "insert.diagram",
  "语义图"
],
[
  "insert.icon",
  "矢量图标"
],
[
  "insert.shape",
  "图形"
],
[
  "insert.table",
  "表格"
],
[
  "insert.wordart",
  "艺术字"
```

```
],
[
  "para.alignCenter",
  "居中"
],
[
  "para.alignLeft",
  "左对齐"
],
[
  "para.alignRight",
  "右对齐"
],
[
  "review.comment",
  "新建批注"
],
[
  "review.commentNext",
  "下一条批注"
],
[
  "review.commentPrevious",
  "上一条批注"
],
[
  "review.commentToggle",
  "显示批注"
],
[
  "review.compare",
  "文档比对"
],
[
  "review.finalize",
  "文档定稿"
],
[
  "review.langToSimplified",
  "繁转简"
],
[
  "review.langToTraditional",
  "简转繁"
],
[
  "review.spell",
  "排版检查"
```

```
],
[
  "slide.background",
  "设置背景"
],
[
  "slide.delete",
  "删除幻灯片"
],
[
  "slide.duplicate",
  "复制幻灯片"
],
[
  "slide.layout",
  "版式"
],
[
  "slide.moveDown",
  "下移"
],
[
  "slide.moveUp",
  "上移"
],
[
  "slide.new",
  "新建幻灯片"
],
[
  "slideshow.current",
  "从当前开始"
],
[
  "slideshow.customShow",
  "自定义放映"
],
[
  "slideshow.presenter",
  "演讲者视图"
],
[
  "slideshow.settings",
  "放映设置"
],
[
  "slideshow.start",
  "从头开始"
```

```
],
[
  "text.bold",
  "加粗"
],
[
  "text.color",
  "字体颜色"
],
[
  "text.italic",
  "斜体"
],
[
  "text.sizeDown",
  "减小字号"
],
[
  "text.sizeUp",
  "增大字号"
],
[
  "text.underline",
  "下划线"
],
[
  "tools.batch",
  "批量工具"
],
[
  "tools.capture",
  "截屏"
],
[
  "tools.ocr",
  "图片转文字"
],
[
  "tools.pageMerge",
  "合并演示文稿"
],
[
  "tools.pageSplit",
  "拆分页面"
],
[
  "tools.record",
  "屏幕录制"
```